

Airbnb 面试通关手册

一本把“怎么想”讲清楚的面经
不只给答案，更给可迁移的设计心法

SZY

2026 年 6 月 30 日

前言

市面上的面经大多是“题目 + 答案”的清单：你照着背，背完了，遇到稍微变形的题就又卡住。这本书想做一件不一样的事——把“怎么想”讲清楚。

Airbnb 的面试有一个鲜明的特点：它不喜欢考冷僻的算法套路，而喜欢考**贴近真实业务的场景题**，并且**会在你写完之后不断追加新需求**。这意味着，单纯记住某道题的最优解几乎没用——面试官真正想看的，是你能不能把问题建模清楚、能不能写出可以继续扩展的代码、能不能在系统设计里讲清楚每一个取舍背后的“为什么”。

所以本书的每一道题、每一个系统，都会围绕四种结构化的“盒子”展开：

- **【设计思想】**（红框）：这道题/这个系统真正考的、可以迁移到别处的心法。
- **【陷阱】**（橙框）：最容易写错、最容易漏的边界。
- **【追问】**（蓝框）：面试官现场最常见的 follow-up，以及怎么接。
- **【要点】**（绿框）：复杂度与速记结论。

如果你只有两周时间，请直接翻到最后一章的冲刺计划，按表执行；如果你有更多时间，建议从头读——因为本书刻意把“设计思想”作为一条主线贯穿算法与系统设计两部分，读完你会发现：防止订单超卖的乐观锁、中位数数据流的对顶堆、搜索系统的分层缓存，本质上都是同一批思维工具的不同投影。

祝面试顺利。

——SZY

目录

前言	i
I 面试全景	1
1 Airbnb 面试全景	2
1.1 整体流程	2
1.2 各轮在考什么	3
1.3 Airbnb 的核心价值观 (Core Values)	3
1.4 时间线与建议	3
II 算法编码	4
2 Airbnb 算法心法	5
2.1 Airbnb 算法题的三个特征	5
2.2 解题的标准动作 (边写边说)	5
2.3 必须烂熟于心的工具箱	6
3 高频题精解	7
3.1 文本左右对齐 (Text Justification)	7
3.2 扁平化嵌套列表迭代器 (Flatten Nested List Iterator)	10
3.3 火星词典 (Alien Dictionary)	12
3.4 组合总和 (Combination Sum)	14
3.5 基于时间的键值存储 (Time Based Key-Value Store)	16
3.6 数据流的中位数 (Find Median from Data Stream)	18
3.7 回文对 (Palindrome Pairs)	20
3.8 滑动谜题 (Sliding Puzzle)	22
3.9 倒水 (Pour Water)	24
3.10 分页去重 (Display Pages / Pagination)	26
3.11 小结：十道题背后的同一套思维工具	29

III 系统设计	30
4 系统设计方法论	31
4.1 五步答题框架	31
4.2 容量估算速查	32
4.3 一致性、CAP 与权衡	32
4.4 画架构图的小建议	32
5 设计订房系统	33
5.1 第一步：需求澄清	33
5.1.1 功能性需求	33
5.1.2 非功能性需求	34
5.2 第二步：容量估算	34
5.3 第三步：高层架构	35
5.4 第四步：数据模型与核心 API	36
5.4.1 数据模型	36
5.4.2 核心 API 契约	37
5.5 第五步（核心难点）：如何防止超卖	38
5.5.1 方案 A：数据库事务 + 悲观锁 (SELECT ... FOR UPDATE)	38
5.5.2 方案 B：乐观并发控制 (version / CAS)	39
5.5.3 方案 C：Redis 分布式锁——能用，但别当成银弹	39
5.5.4 三方案对比与推荐	40
5.6 预订生命周期：状态机与超时回滚	40
5.7 瓶颈、容错与扩展	41
5.7.1 分片 (Sharding)	41
5.7.2 缓存可用性，读写分离	41
5.7.3 限流与熔断	42
5.7.4 容灾与多活	42
6 设计房源搜索	43
6.1 第一步：需求澄清	43
6.1.1 功能性需求	43
6.1.2 非功能性需求	43
6.2 第二步：容量估算	44
6.3 第三步：高层架构	44
6.4 第四步：数据模型与核心 API	45
6.4.1 搜索文档 (Elasticsearch)	45
6.5 第五步（核心难点一）：地理空间索引	46
6.5.1 GeoHash：把经纬度交错编码成字符串前缀	46
6.5.2 S2：把球面映射到立方体的 Hilbert 曲线	46
6.5.3 QuadTree：按密度自适应细分的四叉树	47

6.6	核心难点二：多维过滤与倒排索引	47
6.7	核心难点三：日期可用性如何融入搜索	48
6.7.1	预过滤 (pre-filter) vs 后过滤 (post-filter)	48
6.8	核心难点四：排序、个性化与分层缓存	49
6.8.1	两阶段排序	49
6.8.2	分层缓存	49
6.9	瓶颈、容错与扩展	50
6.9.1	ES 分片与副本	50
6.9.2	近实时索引管线	50
6.9.3	数据新鲜度的取舍	50
7	设计消息与支付系统	52
7.1	第一部分：消息与通知系统	52
7.1.1	第一步：需求澄清	52
7.1.2	第二步：高层架构	53
7.1.3	第三步：推还是拉？	53
7.1.4	第四步：数据模型	53
7.1.5	第五步：不丢、不重、有序	54
7.1.6	扇出 (Fan-out)	55
7.2	第二部分：支付与信任系统	55
7.2.1	第一步：需求澄清	55
7.2.2	第二步：高层架构	56
7.2.3	第三步：核心设计	56
7.2.4	退款、拒付与状态机	57
7.2.5	反欺诈与信任安全	58
7.2.6	对账：最终的安全网	58
8	可复用的分布式设计思想	60
8.1	幂等性 (Idempotency)	60
8.2	强一致并发控制：悲观锁 vs 乐观锁	60
8.3	Saga 与补偿事务	61
8.4	缓存策略与失效	61
8.5	水平分片 (Sharding)	62
8.6	限流与熔断	62
IV	行为面与冲刺	63
9	行为面与文化匹配	64
9.1	STAR 框架	64
9.2	对应四条价值观的故事清单	64
9.3	高频问题与应答要点	65

9.4 把你已有的经历改写成故事	65
10 资源清单与两周冲刺	66
10.1 免费资源清单	66
10.1.1 系统设计	66
10.1.2 算法刷题	66
10.1.3 真实面经	66
10.2 两周冲刺计划	66
10.2.1 第一周：建立框架	67
10.2.2 第二周：查漏 + 模拟	67

Part I

面试全景

Chapter 1

Airbnb 面试全景

在动手刷题之前，先花十分钟把整个流程、各轮的权重、以及 Airbnb 独有的文化考察看清楚。很多技术很强的候选人，恰恰栽在“不知道对方在考什么”上。

1.1 整体流程

Airbnb 软件工程师 (SWE) 的面试通常分为三个阶段：

1. **招聘人电话沟通 (Recruiter Phone Screen)**：聊背景、动机、时间线，轻量，重点是确认匹配度与意向。
2. **技术电话面 (Technical Phone Screen)**：一轮算法编码，约 45 分钟，在共享编辑器 (Coder-Pad) 上完成。
3. **Onsite / 虚拟现场 (4-5 轮)**：
 - **算法编码** ×2：Airbnb 特色是“真实场景题”，常常需要多步迭代、面向对象建模，而不是纯粹的 LeetCode 套路。
 - **系统设计** ×1（资深岗必有）。
 - **跨职能 / 行为面** ×1-2：Airbnb 极度重视文化匹配，这一轮权重很高。

【设计思想】把面试当成一次“协作”，而不是一次“考试”

Airbnb 的工程文化强调协作与沟通。面试官在评估的不只是“你能不能解出来”，而是“我宁愿每天和这个人一起写代码”。因此：主动澄清需求、边写边解释思路、被指出问题时大方修正——这些“软信号”在 Airbnb 的评分卡里权重很高。沉默地闷头写出最优解，反而可能拿不到 strong hire。

1.2 各轮在考什么

轮次	核心考点	Airbnb 侧重
算法编码	数据结构、建模	真实场景、可扩展代码、迭代追加需求
系统设计	分布式架构	双边市场、防超卖、信任与防欺诈、全球规模
行为面	文化与协作	core values 故事、冲突处理、利他

【要点】一句话区分

算法轮考“你写的代码能不能继续长大”；**设计轮**考“你做每个取舍说不说得为什么”；**行为轮**考“你是不是 Airbnb 想要的那种人”。

1.3 Airbnb 的核心价值观 (Core Values)

行为面几乎完全围绕这四条价值观展开，建议**每一条都准备 1-2 个真实的 STAR 故事**（见第四部分）：

- **Champion the Mission**：认同“让人有归属感 (belong anywhere)”的使命。
- **Be a Host**：利他、为他人着想、主动帮助队友。
- **Embrace the Adventure**：拥抱不确定，成长心态，敢啃硬骨头。
- **Every Frame Matters**：注重细节，追求卓越，对自己的产出有主人翁意识。

【陷阱】最容易被低估的一轮

很多技术过硬的候选人把行为面当成“走过场”，临场编故事，结果细节经不起追问而失分。事实上，文化匹配轮在 Airbnb 是**可以一票否决**的。务必提前把故事写下来、讲顺。

1.4 时间线与建议

从初面到 onsite 通常 2-4 周，结果一般一周内出。建议把准备拆成三块并行推进：**算法**（每天 1-2 题，重质不重量）、**系统设计**（吃透 2-3 个核心题）、**行为故事**（写 6-8 个 STAR）。具体节奏见最后一章的两周冲刺计划。

Part II

算法编码

Chapter 2

Airbnb 算法心法

这一章不讲具体题目，只讲“做 Airbnb 算法题时脑子里应该装着什么”。把这套心法装进去，再看下一章的十道题，你会发现它们其实是同一套思维的不同投影。

2.1 Airbnb 算法题的三个特征

1. **偏模拟与建模，少冷门 DP**。Airbnb 爱出“把一段现实逻辑翻译成代码”的题：文本排版、倒水、分页去重。这类题没有花哨算法，但边界极多，考的是你能不能把规则不重不漏地落到代码上。
2. **多步迭代，会追加需求**。面试官常在你写完后说“现在再加一个条件……”。所以第一版就要写得可**扩展**：把规则抽成函数、把状态封进对象，而不是一坨写死的 `if/else`。
3. **重视面向对象设计 (OOD)**。迭代器、分页器、限流器这类题，本质是“设计一个类”，考的是接口划分与封装。

【设计思想】为“被追加需求”而写代码

区分“能 AC 的代码”和“Airbnb 想要的代码”的关键，是**可扩展性**。一个实用的习惯：把“业务规则”从“控制流程”里抽出来。例如分页去重题，不要把“同一 host 不相邻”这条规则硬编码在循环里，而是写成一个 `bool canPlace(item, page)` 判定函数。当面试官追加“VIP host 优先”时，你只需改一个函数，而不是重写整个循环。这一个习惯，往往就是 hire 和 strong hire 的差别。

2.2 解题的标准动作（边写边说）

1. **澄清**：先复述题意、确认输入范围与边界（空输入？重复元素？大小限制？），举一个自己的小样例。
2. **先暴力，再优化**：大方说出暴力解和它的复杂度，再讲怎么优化。面试官想看到你的思考路径，而不是凭空蹦出最优解。
3. **先讲思路，再写码**：达成一致后再动手，避免写到一半推倒重来。

4. **边写边解释**：让面试官跟得上你的每一步。
5. **自测**：写完用样例和边界 case 走一遍，主动发现 bug。

【要点】复杂度报价

写完一定要主动报时间/空间复杂度，并说明瓶颈在哪。这是基本职业素养，不报会扣分。

2.3 必须烂熟于心的工具箱

下面这些是 Airbnb 高频题反复用到的“原子工具”。下一章每道题都会从这个工具箱里取件：

工具	典型用途（对应下一章的题）
哈希表	时间键值存储、回文对、火星词典建图
对顶堆（双堆）	数据流中位数
二分查找	时间键值存储（在有序版本上找 $\leq t$ 的最大值）
回溯	组合总和
BFS / 状态编码	滑动谜题（状态空间搜索）
拓扑排序	火星词典（字符偏序）
栈	嵌套迭代器的惰性展开
模拟 + 规则函数	文本对齐、倒水、分页去重

【追问】“能不能让它支持流式/超大输入？”

这是 Airbnb 最爱的追问方向之一。应答套路：把“一次性加载全部数据”的解法，改造成**迭代器/在线算法**——只保留必要状态。例如中位数题天然就是流式（双堆增量维护）；时间键值存储可以谈“如果数据放不下内存，按 key 分片或落盘 + 二分”。提前想好这条线，追问来了就能从容接住。

【陷阱】只顾最优解，丢了沟通

Airbnb 算法轮里，一个全程沉默、最后甩出最优解的候选人，评分往往不如一个边讲边写、思路清晰但代码略有瑕疵的候选人。**沟通本身就是评分项**。

Chapter 3

高频题精解

如果说大厂的算法面普遍是“在白板上证明你刷过题”，那么 Airbnb 的算法面更像是“在白板上证明你能写出未来同事愿意维护的代码”。这是一个被很多候选人忽视、却决定成败的差别。

Airbnb 的高频题有几条非常稳定的特征。第一，**偏真实场景模拟**：文本排版、倒水、分页、键值存储——这些题目几乎都能在产品里找到对应的影子，而不是凭空构造的数学谜题。第二，**偏多步迭代**：一道题往往不是“给一个输入、求一个输出”就结束，而是要你维护状态、按规则一步步推进，过程本身就是答案。第三，**偏面向对象建模与可扩展性**：迭代器、键值库、分页器，本质都是在问“你能不能设计一个干净的类，让它日后好扩展”。第四，**不爱冷门 DP**：你几乎不会在 Airbnb 的电面里遇到状态压缩 DP 或斜率优化，但你极可能遇到一道看似简单、却在你写完后被追加三次新需求的模拟题。

这就引出 Airbnb 面试最核心的潜规则：**面试官会在你写完后追加新需求**。“如果空格要从右往左分配呢？”“如果嵌套列表可以无限深呢？”“如果同一个 host 的房源也要限制跨页相邻呢？”——他们考的从来不是你能不能写出第一版，而是你的第一版有没有给第二版留出空间。因此，本章每道题除了给出最优解，都会用【设计思想】框提炼**可迁移的心法**，并用【追问】框预演面试官最可能抛出的下一个问题。请把注意力放在“怎么想”和“怎么扩展”上，而不是把代码背下来。

下面逐题精解 10 道 Airbnb 高频题。建议你先盖住题解，自己推一遍思路，再对照阅读。

3.1 文本左右对齐 (Text Justification)

LeetCode 68 | 难度：**Hard**

题意

给定一个单词数组 `words` 和一个最大行宽 `maxWidth`，把这些单词排成若干行，使每一行恰好为 `maxWidth` 个字符，做到**左右两端对齐** (justify)。规则：

- 每行尽量多塞单词；相邻单词间至少一个空格。
- 一行有多个单词时，空格要**尽量均匀**地分配到各单词间隙；若不能均分，则**左侧的间隙分得更多**。
- **最后一行以及只有一个单词的行采用左对齐**（单词间单空格，行尾补空格）。

例: words = ["This", "is", "an", "example", "of", "text", "justification. "], maxWidth = 16, 输出

```
1 "This    is    an"
2 "example of text"
3 "justification. "
```

思路推导

这道题没有任何算法技巧，纯粹是**把人类排版规则翻译成代码**，难点全在边界。不要试图一口气写完，要把问题切成两个正交的子问题：

第一步，**贪心分行**。从左到右扫描，维护“当前行已选单词”和“它们的总长度”。判断单词 w 能否加入当前行的条件是：已有单词长度之和 + 已有单词数（每个单词后至少一个空格）+ $w.size() \leq maxWidth$ 。一旦放不下，就结算当前行、另起一行。这一步保证“每行尽量多塞单词”。

第二步，**对每一行做空格分配**。设当前行有 k 个单词、字母总数为 $letters$ ，则要填充的空格总数为 $maxWidth - letters$ ，要分配到 $k-1$ 个间隙。

$$gap = \left\lfloor \frac{maxWidth - letters}{k - 1} \right\rfloor, \quad extra = (maxWidth - letters) \bmod (k - 1).$$

前 $extra$ 个间隙各多放一个空格（“余数给左边”）。这恰好对应“尽量均匀、不能均分则左侧更多”。

第三步，**两个特例走左对齐**：最后一行、以及任何只有一个单词的行（此时 $k-1=0$ 会除零）。它们用单空格连接，再在行尾补足空格。

把这三步分别写成小逻辑，整道题就清晰了。先分行、再格式化，是处理一切排版题的通用骨架。

【设计思想】把“人类规则”拆成正交的小步骤

这类模拟题的杀手锏是**分治式建模**：先识别出几个互不干扰的子规则（分行 / 行内空格 / 特例左对齐），分别实现、分别验证，再组合。新手最大的错误是想用一个嵌套 if 一次性处理所有情况，结果余数、除零、末行三类边界互相纠缠。记住：**当一段逻辑里出现“尽量均匀 + 余数偏向某侧 + 个别特例”时，几乎一定要拆成独立函数**。这种拆分能力会在系统设计里反复用到。

```
1 class Solution {
2 public:
3     vector<string> fullJustify(vector<string>& words, int maxWidth) {
4         vector<string> result;
5         int n = words.size();
6         int i = 0;
7         while (i < n) {
8             // 第一步：贪心确定本行单词区间 [i, j)
9             int j = i;
```



```
10         int letters = 0; // 本行字母总数 (不含空格)
11         while (j < n &&
12             letters + words[j].size() + (j - i) <= (size_t)maxWidth) {
13             letters += words[j].size();
14             ++j;
15         }
16         int wordCount = j - i; // 本行单词数
17         string line;
18
19         // 第二步 / 第三步: 判断是否为末行或单词行 (左对齐)
20         bool isLastLine = (j == n);
21         if (wordCount == 1 || isLastLine) {
22             for (int k = i; k < j; ++k) {
23                 if (k > i) line += ' ';
24                 line += words[k];
25             }
26             line += string(maxWidth - line.size(), ' '); // 行尾补空格
27         } else {
28             int totalSpaces = maxWidth - letters;
29             int gaps = wordCount - 1;
30             int gap = totalSpaces / gaps; // 每个间隙的基础空格数
31             int extra = totalSpaces % gaps; // 余数, 从左侧间隙开始多给
32             for (int k = i; k < j; ++k) {
33                 line += words[k];
34                 if (k < j - 1) { // 最后一个单词后不加空格
35                     int spaces = gap + (k - i < extra ? 1 : 0);
36                     line += string(spaces, ' ');
37                 }
38             }
39         }
40         result.push_back(line);
41         i = j;
42     }
43     return result;
44 }
45 };
```

【要点】复杂度

设所有单词总字符数为 N 。分行扫描每个单词一次, 格式化每行时填充的字符数总和为 $O(N)$, 故时间 $O(N)$, 额外空间 $O(N)$ (输出本身)。

【陷阱】三类边界

(1) **除零**：单词行 $k=1$ 时 $\text{gaps}=0$ ，必须走左对齐分支，否则 $\text{totalSpaces} / \text{gaps}$ 崩溃。(2) **末行**：即使有多个单词也是左对齐，不要均分空格。(3) **装词判断**的 $(j - i)$ 代表“已有单词各需要一个尾随空格”，容易写成 $(j - i + 1)$ 或漏掉，导致每行少装一个词。

【追问】“如果要求右侧间隙分得更多呢？”

只需把多分配空格的条件从“前 extra 个间隙”改成“后 extra 个间隙”：把判断 $(k - i < \text{extra})$ 改为 $(j - 1 - k \leq \text{extra})$ 之类的对称写法。这正是面试官想看的——你把“余数分配方向”抽象成了一个可调参数，而不是写死在循环里。可以主动提：“我把空格分配策略独立出来，未来无论左偏、右偏还是居中都只改一处。”

3.2 扁平化嵌套列表迭代器 (Flatten Nested List Iterator)

LeetCode 341 | 难度：Medium

题意

给定一个嵌套的整数列表，每个元素要么是一个整数，要么是一个列表（其元素同样可能是整数或列表，可无限嵌套）。实现迭代器类 `NestedIterator`，支持 `hasNext()` 与 `next()`，按深度优先顺序逐个返回所有整数。给定接口：

```
1 class NestedInteger {
2     public:
3         bool isInteger() const;           // 是否为单个整数
4         int getInteger() const;           // 取整数（仅当 isInteger() 为真）
5         const vector<NestedInteger>& getList() const; // 取子列表
6     };

```

例：[[1,1],2,[1,1]] 迭代输出 1 1 2 1 1；[1,[4,[6]]] 输出 1 4 6。

思路推导

有两条路线，面试官想听你比较它们。

路线 A：构造时预处理 (eager)。在构造函数里递归地把整棵树拍扁，存进一个 `vector<int>`，再用一个下标游标遍历。实现最简单，`next/hasNext` 都是 $O(1)$ 。缺点：构造时就把所有数读进内存，若列表极大或后续根本不会遍历完，就浪费了时间和空间——这违背了迭代器“惰性”的初衷。

路线 B：栈式惰性展开 (lazy)。迭代器的精髓是**用时才算**。用一个栈保存“尚未展开的指针”。关键技巧：栈里存的不是值，而是**列表的迭代位置**，这样才能在中途暂停。一个干净的实现是：栈中保存若干 `vector<NestedInteger>` 的“当前指针 + 结束指针”对。每次 `hasNext` 时，反复检查栈顶：若栈顶当前元素是列表，则弹出并把该子列表压栈（展开一层）；若是整数，就停下，说明有下一个值。这样只在需要时展开一层，未触及的深层分支永远不会被计算。

为什么 Airbnb 偏爱这道题？因为它是**纯粹的面向对象 + 迭代器设计题**，没有花哨算法，却能看出你是否理解“接口契约”“惰性求值”“状态封装”。下面给出更易写对的栈式实现：把每个待处理列表用（指针 `begin`，指针 `end`）表示，避免反向压栈。

【设计思想】迭代器 = 把“遍历状态”显式存进对象

递归遍历的状态藏在**函数调用栈**里，函数一返回状态就没了；而迭代器要求“随时暂停、随时继续”，所以必须把那条调用栈**手动搬到一个显式的数据结构（栈）**里。这是“递归转迭代”的通用心法：**凡是需要可暂停/可恢复的遍历，就把隐式调用栈显式化**。同样的思路可用于二叉树中序迭代器、文件系统惰性遍历、协程式生成器。

```

1  class NestedIterator {
2      // 栈中每个元素是一个区间 [cur, end)，表示某层列表尚未处理的部分
3      struct Frame {
4          vector<NestedInteger>::const_iterator cur, end;
5      };
6      vector<Frame> stk;
7
8      // 把栈顶整理到“栈顶区间的当前元素是一个整数”或栈空为止
9      void advance() {
10         while (!stk.empty()) {
11             Frame& top = stk.back();
12             if (top.cur == top.end) {           // 当前层处理完，回退
13                 stk.pop_back();
14                 continue;
15             }
16             if (top.cur->isInteger()) return; // 找到下一个整数，停下
17             // 当前元素是列表：先把指针后移，再展开该子列表（压栈）
18             const auto& lst = top.cur->getList();
19             ++top.cur;                          // 注意：先移动，避免引用失效后重读
20             stk.push_back({lst.begin(), lst.end()});
21         }
22     }
23 public:
24     NestedIterator(vector<NestedInteger>& nestedList) {
25         stk.push_back({nestedList.begin(), nestedList.end()});
26         advance();
27     }
28     int next() {
29         int val = stk.back().cur->getInteger();
30         ++stk.back().cur;    // 消费掉当前整数
31         advance();          // 预备下一个
32         return val;
33     }
34     bool hasNext() {

```

```

35     return !stk.empty();
36 }
37 };

```

【要点】复杂度

设整数总数为 N 、列表节点总数为 M 。整个迭代过程每个节点恰好进栈、出栈一次，总计 $O(N + M)$ ；`next/hasNext` 摊还 $O(1)$ 。空间为栈的最大深度，即嵌套的最大层数 $O(D)$ ，优于 `eager` 方案的 $O(N)$ 。

【陷阱】引用失效与展开顺序

在 `advance` 里展开子列表时，必须先 `++top.cur` 再 `push`。因为 `push_back` 可能触发栈底层 `vector` 扩容，使先前取得的引用 `top` 失效；先移动指针、用局部引用 `lst` 保存子列表（子列表对象本身不在栈内、不会被搬动），就安全了。这是 C++ 迭代器/引用失效的经典坑。

【追问】“如果还要支持 `remove()` 或反向迭代呢？”

栈式方案天然支持惰性，但反向迭代需要双端展开，代价较大；此时可以坦诚指出：“如果访问模式是双向或随机的，惰性栈不再划算，我会退回 `eager` 拍扁成数组，用下标支持 $O(1)$ 随机访问。”这恰好把【设计思想】里 `eager/lazy` 的权衡讲圆——**迭代器的最优实现取决于访问模式**，没有银弹。

3.3 火星词典 (Alien Dictionary)

LeetCode 269 | 难度：Hard

题意

有一种火星语言，使用小写英文字母，但字母表顺序未知。给定一个按该语言字典序升序排列的单词列表 `words`，推断出一种合法的字母表顺序（以字符串返回）。若顺序非法返回空串；若有多解返回任意一个。例：`["wrt", "wrf", "er", "ett", "rftt"]` 则一个合法顺序为 `"wertf"`。

思路推导

关键洞察：相邻两个单词的第一处不同字符，揭示了一条顺序关系。比较 `"wrt"` 和 `"wrf"`，前两个字符相同，第三个 `t` 与 `f` 不同，于是可断定 `t` 排在 `f` 前面 (`t -> f`)。把每对相邻单词都这样比较，就得到一组形如 `a -> b` 的偏序约束。

“从一堆偏序约束中排出一个全序”——这正是**拓扑排序**的定义。于是算法成型：

1. **建图**：节点是出现过的所有字母；对每对相邻单词，找到第一处不同字符 `c1`, `c2`，加一条边 `c1 -> c2`（注意去重，避免重复加入度）。
2. **拓扑排序** (Kahn 算法 / BFS)：把所有入度为 0 的字母入队，逐个出队并追加到答案、削减其后继入度，新变为 0 的入队。

3. **环检测**：若最终排出的字母数少于图中字母总数，说明存在环（约束矛盾），返回空串。

还有一个**致命边界**：如果前一个单词是后一个单词的**前缀且更长**，例如 `["abc", "ab"]`，这在合法字典序里不可能出现（短的应在前），必须判为非法、返回空串。很多人只比较到较短长度就忽略了这一点。

【设计思想】“求一个满足所有约束的顺序” = 拓扑排序

当题目把信息描述成**两两之间的先后/依赖关系**，并要你**排出一个全局顺序**时，几乎都是拓扑排序的伪装：课程表、编译依赖、任务调度、版本升级路径都是同一题。心法分三步：**抽取局部约束 -> 建有向图 -> 拓扑排序并顺便检测环**。难点往往不在排序本身（模板固定），而在“如何从原始输入正确抽取出边”，以及识别那些让图非法的隐藏边界。

```

1  class Solution {
2  public:
3      string alienOrder(vector<string>& words) {
4          unordered_map<char, unordered_set<char>> graph; // 邻接表（去重）
5          unordered_map<char, int> indeg;                // 入度
6
7          // 初始化所有出现过的字母，入度记为 0
8          for (const string& w : words)
9              for (char c : w) indeg[c] = 0;
10
11         // 逐对相邻单词建图
12         for (int i = 0; i + 1 < (int)words.size(); ++i) {
13             const string& a = words[i];
14             const string& b = words[i + 1];
15             int len = min(a.size(), b.size());
16             int j = 0;
17             while (j < len && a[j] == b[j]) ++j;
18             if (j == len) {
19                 // 前缀边界：前者更长却是后者前缀 -> 非法
20                 if (a.size() > b.size()) return "";
21             } else {
22                 // 找到第一处不同：a[j] 在 b[j] 之前
23                 if (!graph[a[j]].count(b[j])) {
24                     graph[a[j]].insert(b[j]);
25                     ++indeg[b[j]];
26                 }
27             }
28         }
29
30         // Kahn 拓扑排序
31         queue<char> q;
32         for (auto& [c, d] : indeg)
33             if (d == 0) q.push(c);

```

```

34
35     string order;
36     while (!q.empty()) {
37         char c = q.front(); q.pop();
38         order += c;
39         for (char nxt : graph[c])
40             if (--indeg[nxt] == 0) q.push(nxt);
41     }
42
43     // 若未排完所有字母，存在环 -> 非法
44     return order.size() == indeg.size() ? order : "";
45 }
46 };

```

【要点】复杂度

设单词总字符数为 C 、不同字母数为 V (≤ 26)、边数为 E 。建图 $O(C)$ ，拓扑排序 $O(V + E)$ 。由于字母至多 26 个，整体近似 $O(C)$ ，空间 $O(V + E)$ 。

【陷阱】两个最常漏的边界

(1) **前缀非法**：["abc", "ab"] 必须返回空串——别只比到 min 长度就收工。(2) **孤立字母**：只在某个单词里出现、不参与任何约束的字母也必须出现在答案里，所以要先用所有字母初始化 indeg，而不是只把建图时遇到的字母入图。(3) **重复边**导致入度被重复 ++，必须用 set 去重，否则该节点入度永远减不到 0，被误判成环。

【追问】“DFS 也能做拓扑排序，你为什么选 BFS？”

两者都对。可答：“Kahn (BFS) 的环检测最直观——排完发现少了节点就是有环，不需要维护三色状态；而且天然支持按入度顺序输出，便于在多解中挑字典序最小解（把队列换成优先队列即可）。DFS 版需要在递归里区分‘正在访问’和‘已完成’两种状态来检测后向边，略易写错。”这番比较正是面试官想听的取舍说明。

3.4 组合总和 (Combination Sum)

LeetCode 39 | 难度：Medium

题意

给定一个无重复正整数数组 candidates 和目标值 target，找出所有使数字和为 target 的组合。同一个数字可以被无限次重复选取。组合不计顺序 ([2,2,3] 与 [2,3,2] 视为同一个)，返回所有不重复组合。例：candidates = [2,3,6,7]，target = 7 则输出 [[2,2,3],[7]]。

思路推导

“找出所有满足条件的组合”是回溯 (backtracking) 的典型信号——我们要枚举一棵决策树的所有合法叶子。核心要回答三个问题：决策是什么？如何避免重复？如何剪枝？

决策结构：在每一步，我们决定“从某个起点开始，选哪个候选数”。由于一个数可重复选，选了 `candidates[i]` 之后，下一层仍然允许再从 `i` 开始（而不是 `i+1`）。

去重：组合不计顺序，若不加约束，`[2,3]` 和 `[3,2]` 会被各算一次。解决办法是**规定选取下标单调不减**：每层递归传入一个起始下标 `start`，只考虑 `candidates[start..]`，绝不回头。这样每个组合只会按“下标升序”这一种方式被生成一次，天然去重。

剪枝：先把 `candidates` 排序。在循环里，一旦 `candidates[i] > remain`（剩余目标），由于后面的数更大，可直接 `break` 整个分支，避免无谓递归。

把这三点装进标准回溯模板：进入时检查 `remain==0` 收集答案；否则从 `start` 起逐个尝试，选入 -> 递归 -> 撤销（回溯）。

【设计思想】回溯三件套：选择、约束、撤销

所有“枚举全部方案”的题都可套同一骨架：**(1) 做选择**（把元素加入当前路径）；**(2) 带着新状态递归**；**(3) 撤销选择**（弹出，恢复现场）。控制**重复**的通用手法是“强加一个单调顺序”——要么下标不减（允许重复取则传 `i`，不允许则传 `i+1`），要么值不减。控制**效率**的通用手法是“排序 + 提前剪枝”。把这三件套刻进肌肉记忆，子集、全排列、组合、分割回文串都是它的变体。

```

1 class Solution {
2     vector<vector<int>> result;
3     vector<int> path;           // 当前组合路径
4
5     void backtrack(vector<int>& cand, int start, int remain) {
6         if (remain == 0) {      // 凑齐目标，收集一个解
7             result.push_back(path);
8             return;
9         }
10        for (int i = start; i < (int)cand.size(); ++i) {
11            if (cand[i] > remain) break; // 排序后剪枝：更大的数都不可能
12            path.push_back(cand[i]);    // 选择
13            // 允许重复选，故下一层仍从 i 开始（不是 i+1）
14            backtrack(cand, i, remain - cand[i]);
15            path.pop_back();           // 撤销选择
16        }
17    }
18 public:
19     vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
20         sort(candidates.begin(), candidates.end()); // 排序以便剪枝
21         backtrack(candidates, 0, target);
22         return result;
    }

```



```
23     }  
24 };
```

【要点】复杂度

设解的个数为 S 、每个解平均长度为 L 。回溯树的节点数受解的数量与目标深度约束，复制每个解需 $O(L)$ ，总体可粗略记为 $O(S \cdot L)$ 量级（指数级上界 $O(N^{T/m})$ ，其中 m 为最小候选值）；递归深度 $O(T/m)$ ，额外空间 $O(L)$ 。

【追问】“如果每个数字最多用一次（LC 40），且数组含重复元素呢？”

两处改动：（1）递归时传 $i+1$ 而非 i ，禁止重复使用同一元素；（2）同一层去重——在 `for` 循环里加 `if (i > start && cand[i] == cand[i-1]) continue;`，跳过本层重复值，避免生成重复组合。能当场说出“传 i 还是 $i+1$ ”和“层内去重”这两个旋钮，说明你真正理解了回溯的去重机制，而不是背模板。

3.5 基于时间的键值存储 (Time Based Key-Value Store)

LeetCode 981 | 难度：Medium

题意

设计 `TimeMap` 类，支持：

- `set(key, value, timestamp)`：存储一条带时间戳的键值。
- `get(key, timestamp)`：返回该 `key` 在时间戳不超过 `timestamp` 的所有写入中，时间戳最大的那个 `value`；若不存在返回空串。

保证对同一 `key` 的 `set` 调用其 `timestamp` 严格递增。例：`set("foo", "bar", 1); get("foo", 1) -> "bar"; get("foo", 3) -> "bar"; set("foo", "bar2", 4); get("foo", 4) -> "bar2"。`

思路推导

拆成两层。外层：不同 `key` 互不相干，用一个 `unordered_map<string, ...>` 把每个 `key` 映射到它自己的时间序列。内层：对单个 `key`，`get` 要做的是“在一串时间戳里找不超过 `t` 的最大者”，这是有序数据上的查找。

注意题目的金句：“同一 `key` 的 `timestamp` 严格递增”。这意味着每个 `key` 的时间序列天然按时间戳有序，我们只需把 `(timestamp, value)` 顺序 `push_back` 进一个 `vector`，它就是排好序的。于是 `get` 变成经典的二分查找：找最后一个 `timestamp <= t` 的元素。

用 `upper_bound` 最干净：它返回第一个严格大于 `t` 的位置 `it`，那么 `it` 前一个元素就是我们想要的“不超过 `t` 的最大时间戳”。若 `it` 已是开头，说明所有记录都比 `t` 晚，返回空串。为了能对 `(timestamp, value)` 的 `vector` 用 `upper_bound` 且只比较时间戳，可自定义比较器，或干脆只对时间戳数组手写二分。下面实现把时间戳和值分别讨论，逻辑最直白。

【设计思想】“最近不超过 X” = 有序结构上的二分（前驱查询）

凡是出现“小于等于某值中的最大”“大于等于某值中的最小”这类前驱/后继查询，第一反应是把数据维护成有序结构再二分（有序数组 + lower/upper_bound，或平衡树 map::lower_bound）。本题更进一步给了“写入时间戳递增”的福利，省去了排序，直接 push_back 即有序——读题时要敏锐捕捉这种“输入自带有序性”的暗示，它常常把 $O(n \log n)$ 降到 $O(\log n)$ 。

```
1 class TimeMap {
2     // 每个 key 对应一个按 timestamp 递增的 (timestamp, value) 序列
3     unordered_map<string, vector<pair<int, string>>> store;
4 public:
5     TimeMap() {}
6
7     void set(string key, string value, int timestamp) {
8         // 因 timestamp 严格递增，直接尾插即保持有序
9         store[key].push_back({timestamp, value});
10    }
11
12    string get(string key, int timestamp) {
13        auto it = store.find(key);
14        if (it == store.end()) return "";
15        const auto& arr = it->second;
16        // 手写二分：找第一个 timestamp 严格大于查询值的位置（等价 upper_bound）
17        int lo = 0, hi = arr.size();    // 在 [lo, hi) 上二分
18        while (lo < hi) {
19            int mid = lo + (hi - lo) / 2;
20            if (arr[mid].first <= timestamp) lo = mid + 1; // 仍可行，右移
21            else hi = mid;
22        }
23        // 循环结束后 lo 为第一个 > timestamp 的下标
24        if (lo == 0) return "";           // 没有任何记录 <= timestamp
25        return arr[lo - 1].second;       // 前一个即“不超过 t 的最大者”
26    }
27};
```

【要点】复杂度

set 摊还 $O(1)$ （尾插）。get 为单 key 序列上的二分， $O(\log n)$ ，其中 n 为该 key 的写入次数。总空间 $O(N)$ ， N 为所有写入总数。

【陷阱】二分边界与“严格大于”

要找的是“ $\leq t$ 的最大”，对应 `upper_bound`（第一个 $> t$ ）再退一位，而不是 `lower_bound`（第一个 $\geq t$ ）。若误用 `lower_bound`，当存在恰好等于 `t` 的时间戳时会少取一位、取到前一个值。手写二分时记牢不变式：循环结束 `lo` 指向第一个严格大于 `t` 的位置，答案在 `lo-1`；`lo==0` 表示无解。

【追问】“如果 `set` 的时间戳不保证递增了呢？”

两条路：（1）改用 `std::map<int, string>`（红黑树）按时间戳排序，`get` 用 `upper_bound` 仍是 $O(\log n)$ ，`set` 变 $O(\log n)$ ；（2）仍用 `vector` 但 `set` 时插入到正确位置（ $O(n)$ ）或先收集后排序。可主动权衡：“如果写多读少且乱序，我倾向 `map`；如果写入基本有序、偶有乱序，`vector` + 偶尔排序的均摊成本更低。”

3.6 数据流的中位数 (Find Median from Data Stream)

LeetCode 295 | 难度：**Hard**

题意

设计数据结构支持两种操作：`addNum(num)` 不断加入整数；`findMedian()` 返回当前所有已加入数字的中位数（偶数个时为中间两数的平均）。例：依次 `addNum(1)`、`addNum(2)` 则 `findMedian()` -> 1.5；再 `addNum(3)` 则 `findMedian()` -> 2。

思路推导

朴素做法是维护一个有序数组，`findMedian` $O(1)$ 但 `addNum` 要插入到正确位置， $O(n)$ ；在数据流场景下太慢。我们想要的是“随时拿到中间元素”，却不需要整个序列完全有序——只要能快速取到“中间那一两个”。

这引出经典的**对顶堆 (two heaps)** 结构：把数据从中间劈成两半。

- **大顶堆** `lo` 存较小的一半，堆顶是这半的最大值；
- **小顶堆** `hi` 存较大的一半，堆顶是这半的最小值。

只要维持两条不变式：（1）`lo` 里每个数 $\leq$ `hi` 里每个数；（2）两堆大小相差不超过 1（约定 `lo` 至多比 `hi` 多一个）。那么中位数就在两个堆顶之间：总数为奇则取 `lo` 堆顶；为偶则取两堆顶的平均。这两个堆顶恰是“中间的一两个元素”， $O(1)$ 可得。

`addNum` 的维护：先把新数放进 `lo`（并把 `lo` 堆顶弹给 `hi`，保证跨堆有序），再做**再平衡**——若 `hi` 比 `lo` 多，则把 `hi` 堆顶移回 `lo`。这样两条不变式始终成立。每次操作只涉及常数次堆的弹出/插入， $O(\log n)$ 。

【设计思想】对顶堆：当你只关心“中间/第 k 个”而非全序

若问题需要动态地、反复地访问某个分位点（中位数、第 k 大、滑动窗口中值），不要去维护完整有序序列。用两个堆把数据从关注点处一分为二：一侧大顶堆、一侧小顶堆，让关注的元素永远浮在两个堆顶。这个“在临界点处对半切、用两个相反方向的堆夹住它”的思想，是处理流式分位数问题的标准武器，也能推广到“维护滑动窗口中位数”等变体。

```

1  class MedianFinder {
2      priority_queue<int> lo; // 大顶堆：较小的一半
3      priority_queue<int, vector<int>, greater<int>> hi; // 小顶堆：较大的一半
4  public:
5      MedianFinder() {}
6
7      void addNum(int num) {
8          // 1) 先入大顶堆，再把其最大值让给小顶堆，保证 lo 所有元素 <= hi
9          lo.push(num);
10         hi.push(lo.top());
11         lo.pop();
12         // 2) 再平衡：约定 lo 的规模 >= hi，且至多多 1
13         if (hi.size() > lo.size()) {
14             lo.push(hi.top());
15             hi.pop();
16         }
17     }
18
19     double findMedian() {
20         if (lo.size() > hi.size()) // 总数为奇，中位数在 lo 堆顶
21             return lo.top();
22         // 总数为偶，取两堆顶平均（注意转 double 避免整型溢出/截断）
23         return (lo.top() + (double)hi.top()) / 2.0;
24     }
25 };

```

【要点】复杂度

addNum 涉及常数堆操作， $O(\log n)$ ；findMedian 仅读两个堆顶， $O(1)$ 。空间 $O(n)$ 。

【陷阱】平衡方向与求和溢出

(1)再平衡只需一个方向：因为入队套路固定（先进 lo 再吐给 hi），只可能出现 hi 比 lo 多的情况，故只判一个方向；若两个方向都判反而易写出震荡。(2)偶数取平均时 lo.top()+hi.top() 在两者都接近 INT_MAX 时会整型溢出，务必先转 double 再相加。

【追问】“如果数字都在 0..100 的小范围内，能做到 $O(1)$ 吗？”

能。改用**计数桶**：开一个长度 101 的频次数组，`addNum` $O(1)$ 计数，`findMedian` 时累加频次找到中位位置，固定 $O(100)=O(1)$ 。当数值域很小，桶/计数往往碾压通用堆方案。能根据“数据范围”这一约束切换到更专用的数据结构，是面试官眼里的加分点。另一个常见追问：“99% 的查询只要中位数附近”——可答可缓存上次结果并惰性更新。

3.7 回文对 (Palindrome Pairs)

LeetCode 336 | 难度：**Hard**

题意

给定一组**互不相同**的单词 `words`，找出所有下标对 $(i, j) (i \neq j)$ 使得 `words[i] + words[j]` 拼接后是回文串。返回所有这样的下标对。例：`["abcd", "dcba", "lls", "s", "sssll"]` 则输出 `[[0,1],[1,0],[3,2],[2,4]]`（如 `words[3]+words[2]="s"+"lls"="slls"` 是回文）。

思路推导

暴力是 $O(n^2 \cdot k)$ ：枚举每一对、各判一次回文（ k 为词长）。我们要把“配对”这一维用**哈希**降下来。

核心洞察：固定单词 `w`，问“哪个单词 `x` 能让 `w+x` 或 `x+w` 成为回文”。把 `w` 在某个切点切成两段 `left = w[0..i]` 与 `right = w[i..]`。分两种情况：

1. 若 `left` 本身是回文，那么只要存在另一个单词等于 `reverse(right)`，把它接在 `w` 前面，即 `x + w`，整体就是回文。因为 `x = reverse(right)`，`x + left + right`：`x` 与 `right` 互为镜像、中间 `left` 自回文。
2. 若 `right` 本身是回文，那么只要存在单词等于 `reverse(left)`，把它接在 `w` 后面，即 `w + x`，整体回文。

于是算法：把所有单词的**反转串**建一个哈希表 `reverse(word) -> 下标`。对每个单词 `w`，枚举所有切点 `i in [0..len]`，按上面两种情况查表。注意处理空串与避免自配对、避免重复计数（在切点遍历里对 `i==0` 的情形只在一种分支处理，防止同一对被记两次）。

【设计思想】把“配对搜索”转成“查表”：拆分 + 反转 + 哈希

$O(n^2)$ 的配对枚举常能被**哈希预处理**砍掉一维：与其问“这两个能不能配”，不如对每个元素算出它需要的‘另一半’长什么样，再去表里查那一半是否存在（两数之和、回文对都是此模式）。本题的额外巧思是在**元素内部枚举切点**，把一个长匹配拆成“一段自满足 + 另一段去表里找镜像”。这种“拆成‘自洽部分 + 待匹配部分’”的分解，是字符串/前后缀类难题的通用解法（Trie 是它的另一种载体）。

```
1 class Solution {
2     bool isPalindrome(const string& s, int l, int r) { // 判断 s[l..r] 是否回文
```

```

3         while (l < r) if (s[l++] != s[r--]) return false;
4         return true;
5     }
6 public:
7     vector<vector<int>> palindromePairs(vector<string>& words) {
8         unordered_map<string, int> rev;    // reverse(word) -> 其下标
9         for (int i = 0; i < (int)words.size(); ++i) {
10             string r(words[i].rbegin(), words[i].rend());
11             rev[r] = i;
12         }
13         vector<vector<int>> result;
14         for (int i = 0; i < (int)words.size(); ++i) {
15             const string& w = words[i];
16             int n = w.size();
17             for (int j = 0; j <= n; ++j) {          // 枚举切点 [0..n]
18                 // 情况一: 前缀 w[0..j] 回文 -> 找等于 reverse(w[j..]) 的词接前面
19                 if (isPalindrome(w, 0, j - 1)) {
20                     string suf = w.substr(j);
21                     string need(suf.rbegin(), suf.rend());
22                     auto it = rev.find(need);
23                     if (it != rev.end() && it->second != i)
24                         result.push_back({it->second, i});
25                 }
26                 // 情况二: 后缀 w[j..] 回文 -> 找等于 reverse(w[0..j]) 的词接后面
27                 // j < n 防止与情况一在整词回文时重复计数 (j==n 已在情况一处理)
28                 if (j < n && isPalindrome(w, j, n - 1)) {
29                     string pre = w.substr(0, j);
30                     string need(pre.rbegin(), pre.rend());
31                     auto it = rev.find(need);
32                     if (it != rev.end() && it->second != i)
33                         result.push_back({i, it->second});
34                 }
35             }
36         }
37         return result;
38     }
39 };

```

【要点】复杂度

设单词数 n 、最大词长 k 。建表 $O(nk)$ 。主循环对每个词枚举 $O(k)$ 个切点，每个切点做 $O(k)$ 的回文判断与子串/哈希操作，故 $O(nk^2)$ 。空间 $O(nk)$ 。相比暴力 $O(n^2k)$ 大幅改进 ($k \ll n$ 时尤甚)。

【陷阱】重复计数与空串

(1) **重复**: 当整个词本身回文时, 情况一 ($j==n$) 和情况二 ($j==0$) 会把同一对算两次, 故用 $j < n$ 把情况二的端点排除。(2) **空串**: 若 words 含空串 "", 它能与任意回文词配对, 切点遍历到端点时自然覆盖, 但要确保回文判断对空区间返回真 ($l>r$ 时返回 true)。(3) 别忘了 `it->second != i` 排除自配对。

【追问】“能否用 Trie 把复杂度降到更优?”

可以。把所有单词的**反转串**插入 Trie, 并在每个节点记录“以此为前缀、其剩余后缀为回文的单词下标”。查询某词 w 时沿 Trie 走, 遇到完整匹配且剩余部分回文即成对。这样把每次切点的哈希查找均摊为沿树下行, 整体仍约 $O(nk^2)$ 但常数更小、且利于前缀复用、更省内存。面试中能说清“哈希版好写、Trie 版更省内存”的取舍即可, 通常不必现场写完 Trie。

3.8 滑动谜题 (Sliding Puzzle)

LeetCode 773 | 难度: **Hard**

题意

一个 2×3 的棋盘, 格子里是 0..5, 其中 0 表示空格。每次可把 0 与其上下左右相邻的数字交换。求把棋盘变成目标 $[[1,2,3],[4,5,0]]$ 所需的**最少移动步数**; 若无法达成返回 -1。例: $[[1,2,3],[4,0,5]]$ 只需 1 步 (交换 0 与 5) 则返回 1; $[[1,2,3],[5,4,0]]$ 返回 -1。

思路推导

“求最少步数”且“每步代价相同”——这是**无权图最短路**, 标准答案是 **BFS**。难点不在 BFS 本身, 而在如何把“棋盘状态”变成可入队、可去重的对象。

状态编码: 2×3 棋盘共 6 个格子, 把它按行展开成长度 6 的字符串, 例如 $[[1,2,3],[4,0,5]]$ 编码为 "123405"。字符串可直接放进 `unordered_set` 做 visited 判重, 目标状态是 "123450"。

邻接关系: 在一维字符串里, 0 所在下标 z 能和哪些下标交换? 把二维相邻预先压成一张固定的邻接表: 下标 0 的邻居是 {1,3}, 1 是 {0,2,4}, 2 是 {1,5}, 3 是 {0,4}, 4 是 {1,3,5}, 5 是 {2,4}。每次找到 0 的位置, 按邻接表交换生成新状态。

于是就是教科书 BFS: 起点状态入队, 逐层扩展, 第一次到达 "123450" 时的层数即答案; 队列空仍未达则返回 -1。状态空间至多 $6! = 720$ 种, 极小, BFS 秒解。

【设计思想】最短步数 + 状态转移 = 状态空间 BFS

当题目是“从初始局面经过若干等价代价的操作, 最少多少步到达目标”, 把每个**局面看作图的一个节点**、每个**合法操作看作一条边**, 答案就是 BFS 最短路。三个关键动作: (1) **设计紧凑可哈希的状态编码** (本题用字符串); (2) **写出状态转移 (邻接)**; (3) **用 visited 去重防止指数爆炸**。八数码、华容道、倒水量杯、最少操作变换字符串, 全是这一模式。状态编码的优劣直接决定能不能跑得动。

```

1  class Solution {
2  public:
3      int slidingPuzzle(vector<vector<int>>& board) {
4          // 0 在一维下标处可交换的邻居 (由 2x3 相邻关系压平而来)
5          const vector<vector<int>> neighbors = {
6              {1, 3}, {0, 2, 4}, {1, 5}, {0, 4}, {1, 3, 5}, {2, 4}
7          };
8          string start, target = "123450";
9          for (auto& row : board)
10             for (int x : row) start += ('0' + x);
11
12         unordered_set<string> visited{start};
13         queue<string> q;
14         q.push(start);
15         int steps = 0;
16         while (!q.empty()) {
17             int sz = q.size();
18             while (sz--) { // 逐层扩展
19                 string cur = q.front(); q.pop();
20                 if (cur == target) return steps;
21                 int z = cur.find('0'); // 空格位置
22                 for (int nb : neighbors[z]) {
23                     string nxt = cur;
24                     swap(nxt[z], nxt[nb]); // 交换 0 与邻居
25                     if (!visited.count(nxt)) {
26                         visited.insert(nxt);
27                         q.push(nxt);
28                     }
29                 }
30             }
31             ++steps;
32         }
33         return -1; // 无法达到目标
34     }
35 };

```

【要点】复杂度

状态空间至多 $6! = 720$ 个排列，每个状态扩展常数条边，故时间与空间均为 $O((R \cdot C)!)$ 量级，本题即 $O(720)$ ，实际瞬时完成。

【陷阱】一维邻接表别写错

把二维相邻硬编码成一维邻接表时极易把下标算错（尤其跨行的 0/3、2/5 不相邻）。稳妥做法是按 $r=z/C$, $c=z\%C$ 现算四方向并过滤越界，避免手抄邻接表出错；本题盘面固定，预存邻接表更快，但务必逐项核对。另外 **visited** 要在入队时立刻标记，不要等出队再标，否则同一状态会被重复入队、可能超时。

【追问】“如果是 3×3 八数码、状态空间更大呢？”

3×3 有 $9! = 362880$ 个状态，BFS 仍可行但偏慢。可升级为 **A* 搜索**：用“各数字到目标位置的曼哈顿距离之和”作启发函数（它是可采纳的下界），用优先队列按 已走步数 + 启发值 扩展，能大幅减少访问状态数。还可使用 **双向 BFS** 从起点与终点同时搜、在中间相遇。能说出“BFS 保证最优但盲目，A* 用启发剪枝”这层关系即达标。

3.9 倒水 (Pour Water)

LeetCode 755 | 难度：Medium

题意

给定一维高度数组 `heights` 表示地形，从下标 `K` 正上方依次倒下 `V` 单位水，每个单位水滴遵循物理规则下落与流动，求最终地形 + 水的高度数组。单滴水规则（优先级从高到低）：

1. 若它能往左流到一个更低处（沿途不上升、最终落点严格低于当前位置），就流到左边最近的最低点；
2. 否则若能往右流到更低处，就流到右边最近的最低点；
3. 否则停在原地 `K` 上方。

“高度”指地形高度加已有水高度之和。例：`heights=[2,1,1,2,1,2,2]`, `V=4`, `K=3` 则结果为 `[2,2,2,3,2,2,2]`。

思路推导

这是 Airbnb 最经典的物理模拟题，考点不是算法而是把自然语言规则精确翻译成代码、并处理好优先级与平台（平地）情形。最稳的策略是一次只模拟一滴水，重复 `V` 次；每滴独立、互不干扰，远比想批量处理简单可靠。

对单滴水，从落点 `cur = K` 开始，分别尝试左、右两个方向，规则完全对称：

向某方向探流：从 `cur` 沿该方向逐格前进，维护一个“候选落点”`best`（初值为 `cur`）。前进时：若下一格高度大于当前 `best` 高度，说明遇到上坡，水翻不过去，停止探索；若下一格高度小于 `best` 高度，把 `best` 更新为这一格（找到了更低的落点）；若相等（平地），继续走但暂不更新 `best`（保证“流到最近的最低点”——多个等高最低点取最靠近 `cur` 的那个，因为我们只在严格更低时才更新 `best`）。

最终：若向左探得 `best != cur`（左边确有更低落点），水落在左 `best`；否则同样探右；都不行就落在 `cur`。把对应位置高度 `+1`，进入下一滴。

把“探一个方向”写成一个带方向参数 `dir in -1,+1` 的函数，左右复用同一份逻辑，是这道题写干净的关键。

【设计思想】模拟题：固定“最小可独立单元”，把规则翻译成对称的小函数

面对物理/业务模拟，先找到**最小可独立处理的单元**（这里是“一滴水”），逐个处理而非整体处理，复杂度换来正确性。再把**对称的规则**（向左/向右流动镜像一致）抽成**带方向参数的同一函数**，避免左右两套近乎重复、却各藏一个 off-by-one 的代码。模拟题的失分点几乎全在“规则优先级”和“平地/边界”的细节，把规则逐条写成注释、对照实现，是稳过的笨办法也是好办法。

```

1  class Solution {
2  public:
3      vector<int> pourWater(vector<int>& heights, int V, int K) {
4          int n = heights.size();
5          // 逐滴模拟，每滴互不影响
6          for (int drop = 0; drop < V; ++drop) {
7              int cur = K;
8              // 依次尝试方向：先左(-1)后右(+1)，符合题目优先级
9              for (int dir : {-1, 1}) {
10                 int best = cur;           // 该方向上的最佳落点
11                 int p = cur;
12                 while (p + dir >= 0 && p + dir < n &&
13                     heights[p + dir] <= heights[p]) {
14                     p += dir;
15                     if (heights[p] < heights[best])
16                         best = p;         // 仅在严格更低时更新，取最近最低点
17                 }
18                 if (best != cur) {         // 此方向找到更低落点
19                     cur = best;
20                     break;                // 左侧成功则不再看右侧
21                 }
22             }
23             ++heights[cur];               // 水滴最终停在 cur
24         }
25         return heights;
26     }
27 };

```

【要点】复杂度

每滴水最坏向某方向走 $O(n)$ ，共 V 滴，故时间 $O(V \cdot n)$ ，原地修改、额外空间 $O(1)$ 。

【陷阱】平地“最近最低点”与方向优先级

(1) **平地处理**：探流时遇到等高格要**继续走但不更新 best**，否则会停在远处的等高点而非最近的最低点；只有严格更低才更新 best，保证取到“最近的最低”。(2) **优先级**：必须先彻底判左、左不行才判右，不能左右一起比；代码用 `for dir in {-1,1} ... break` 显式保证顺序。(3) 流动条件是**相对前一格不上升** (`heights[p+dir] <= heights[p]`)，而不是相对起点 cur，写错会让水翻越本不该翻越的小坡。

【追问】“如果 V 很大 (百万级)，如何加速？”

逐滴 $O(Vn)$ 会退化。可观察到水会先填满 K 周围的“盆地”，可用**批量灌水**：在当前左右最近的“墙”之间求出可容纳的水量，一次性灌到下一个临界水位，把 V 个单位分摊处理，类似“接雨水”的双指针/分层填充。能指出“单滴模拟正确但不可扩展，批量按水位填充可降到接近线性”，正是面试官期待的可扩展性思考。

3.10 分页去重 (Display Pages / Pagination)

LeetCode Airbnb 高频 OOD | 难度：**Medium**

题意

给定一组房源记录，每条形如 `"hostId,score,city,..."` (逗号分隔, 第一个字段是 `hostId`, 第二个是评分 `score`)。要求把它们分页输出，每页 12 条，规则：

- 总体按 **score 降序** 排列；
- 同一页内，同一个 **hostId** 的房源不能出现两次（即一页里 host 不重复）；
- 若在某一页中，剩余还没用过的记录里再也找不到“host 未在本页出现过”的记录（即一整轮扫描都无法继续满足约束），则**放宽限制**，允许把剩下的记录按序填入，直到这一页满或数据耗尽，然后开新页。

输出最终的记录顺序（页与页之间可用空行或分隔符标识）。这是 Airbnb 经典的 OOD/业务模拟题，重点在**把业务规则建模成可扩展的分页器**。

思路推导

先厘清数据流：输入已能按 `score` 降序排序（或我们先排序）。难点是“同页 host 不重复，且无法满足时放宽”。

朴素思路：每开一页，维护一个 `usedHosts` 集合，从头扫描剩余记录，遇到 host 不在集合里的就选入、加入集合、从剩余列表删除；扫到 12 条或扫完一轮为止。若某轮一条都没选上（剩余全是已用 host），就放宽：直接按序填满本页。问题：每次从头扫 + 删除元素，复杂度高且代码乱。

更干净的队列模型：把按 `score` 排好序的记录放进一个队列 q（队首 `score` 最高）。逐条出队尝试放入当前页：

- 若当前页 `usedHosts` 不含此 `host` (或已处于“放宽”状态), 就把它放入当前页, 记录其 `host`;
- 否则 (`host` 冲突且未放宽), 暂存到一个 `skipped` 缓冲队列, 留待后续页。

何时翻页或放宽? 维护“自上次成功放入以来连续跳过的计数”或用更稳的判定: 当我们遍历完一轮主队列仍未填满本页, 说明剩下的全是冲突 `host`, 则进入放宽模式把 `skipped` 直接灌入。最稳妥、最易讲清的实现是每页独立处理: 开一页 -> 用一个 `usedHosts` -> 遍历主队列, 能放就放、不能放就留; 若一整轮没放进任何新记录 (且页未滿), 则放宽, 把队首直接放入。页满 12 条则结算开新页。

下面给出面向对象的 `Paginator`: 把“记录解析”“一页的装填规则”“放宽策略”分离, 便于将来扩展 (改页大小、改去重键、改放宽策略)。我们用一个 `deque` 作主队列、按顺序尝试, 配合 `usedHosts`; 当遍历指针走完一圈没有任何放入时触发放宽。

【设计思想】业务规则建模: 把“数据流 + 约束 + 降级策略”拆成可替换的部件

OOD 模拟题考的不是算法, 而是能不能把业务规则拆成正交、可替换的部件, 因为面试官一定会追加新需求。本题至少有四个可变点: 页大小、去重键 (今天是 `hostId`, 明天可能是 `city`)、排序键 (`score`)、降级/放宽策略。好的设计把它们各自参数化: 去重键抽成一个“从记录提取 `key` 的函数”, 放宽策略抽成一个独立判断。这样当面试官说“现在改成 `city` 不重复、每页 10 条、放宽时优先同城”, 你只改一两个部件而非重写。记住: Airbnb 的 OOD 题, 可扩展性就是正确答案的一部分。

```

1  class Paginator {
2      int pageSize;
3      // 从一条记录中提取去重键 (此处为 hostId)。抽成函数便于将来替换去重维度
4      static string extractHostId(const string& record) {
5          return record.substr(0, record.find(','));
6      }
7  public:
8      explicit Paginator(int pageSize = 12) : pageSize(pageSize) {}
9
10     // 输入已按 score 降序排好; 返回分页后的记录顺序 (外层 vector 为各页)
11     vector<vector<string>> paginate(const vector<string>& sortedRecords) {
12         deque<string> q(sortedRecords.begin(), sortedRecords.end());
13         vector<vector<string>> pages;
14
15         while (!q.empty()) {
16             vector<string> page;                // 当前页
17             unordered_set<string> usedHosts;    // 本页已出现的 host
18             int scanned = 0;                    // 本轮已检查的记录数
19             int queueLenAtRoundStart = q.size(); // 用于判断“扫完一轮无进展”
20             bool relaxed = false;               // 是否已放宽 host 去重约束
21
22             while ((int)page.size() < pageSize && !q.empty()) {

```

```

23         string record = q.front(); q.pop_front();
24         string host = extractHostId(record);
25
26         if (relaxed || !usedHosts.count(host)) {
27             // 可以放入本页
28             page.push_back(record);
29             usedHosts.insert(host);
30             scanned = 0; // 有进展, 重置“连续跳过”计数
31             queueLenAtRoundStart = q.size();
32         } else {
33             // host 冲突: 暂时放回队尾, 留待后续页
34             q.push_back(record);
35             ++scanned;
36             // 若一整轮 (队列原长度) 都没放进任何记录, 则放宽约束
37             if (scanned >= queueLenAtRoundStart)
38                 relaxed = true;
39         }
40     }
41     pages.push_back(page);
42 }
43 return pages;
44 }
45 };

```

【要点】复杂度

排序 $O(N \log N)$ (若输入未排序)。分页阶段每条记录被放入一次、最多被“放回队尾”常数轮, 整体近似 $O(N)$ 到 $O(N \cdot P)$ (P 为页数, 放宽判定带来的额外扫描有限)。空间 $O(N)$ 。

【陷阱】放宽时机与死循环

最危险的 bug 是放回队尾导致死循环: 当本页已满或剩余全是冲突 host 却没有放宽开关时, 记录会被无限放回。必须有一个明确的“扫完一轮无进展即放宽”的判据 (本实现用 `scanned >= queueLenAtRoundStart`)。另一个易错点: 放宽是针对当前页的局部状态, 开新页时要重置 `relaxed` 与 `usedHosts`, 否则后续页会无谓地丢失去重约束。

【追问】“现在改成按 city 去重、每页 10 条、放宽时优先同城; 还要支持流式输入”

这正是 Airbnb 式的连环追问, 逐条应答: (1) **改去重键**: 把 `extractHostId` 泛化为可注入的 `keyExtractor` (如 `std::function<string(const string&>`), 构造时传入“取 city”的提取器即可, 主逻辑一行不改。(2) **改页大小**: 已是构造参数。(3) **放宽策略**: 把放宽时的选取逻辑抽成策略对象 (`RelaxPolicy`), 默认“按序”, 可替换为“优先同城”。(4) **流式输入**: 把一次性 `vector` 改为逐页惰性生成 (`nextPage()` 迭代器接口), 内部用堆维护 `top-score`、按需取数, 避免一次加载全部。能把“四个可变点各自参数化”讲清楚, 就完整命中了这道题

的考点——可扩展的业务建模。

3.11 小结：十道题背后的同一套思维工具

回看这十道题，会发现它们反复调用的是同一批可迁移的“思维工具”：

思维工具	代表题	触发信号
分治式规则建模	68 文本对齐、755 倒水、Pagination	“把人类/业务规则翻译成代码”
迭代器 / 状态显式化	341 嵌套迭代器	“可暂停、可恢复的遍历”
拓扑排序	269 火星词典	“从两两先后约束排出全序”
回溯三件套	39 组合总和	“枚举所有满足条件的方案”
有序结构 + 二分	981 时间键值	“小于等于某值中的最大”
对顶堆	295 数据流中位数	“动态访问中位/第 k 个”
拆分 + 哈希配对	336 回文对	“把 $O(n^2)$ 配对降一维”
状态空间 BFS	773 滑动谜题	“最少步数 + 等价代价转移”
可扩展 OOD 建模	Pagination	“面试官会追加新需求”

把这些信号刻进直觉，比记住任何一道题的代码都重要。Airbnb 面试的下半场往往不是“你会不会做这道题”，而是“当我把题改成另一个样子，你能不能稳稳接住”。下一章我们进入系统设计，你会看到乐观锁、分层缓存、对顶堆式的负载均衡——它们和本章的思维工具，其实是同一批武器在更大尺度上的投影。

Part III

系统设计

Chapter 4

系统设计方法论

系统设计轮没有标准答案，但有标准打法。这一章给你一套可以套用在任何题目上的五步框架，以及背后的估算与一致性基础。后面三章（订房、搜索、消息/支付）都会严格按这套框架来组织。

4.1 五步答题框架

1. **澄清需求 (5 分钟)**：别急着画图。先问清楚——
 - 功能性：核心用例是什么？哪些先做、哪些可以砍？
 - 非功能性：日活/QPS 量级？读写比？延迟要求 (p95/p99)？一致性要求（强一致还是最终一致即可）？
2. **容量估算 (3 分钟)**：用数量级估算 QPS、存储、带宽。目的是为后面的分片、缓存决策提供依据，不是为了精确。
3. **高层架构 (8 分钟)**：画出主要组件——客户端、API 网关、服务、缓存、数据库、消息队列、搜索引擎。强调解耦与可扩展。
4. **数据模型 + API (8 分钟)**：给出关键实体的表结构和核心 API 契约。
5. **深挖 + 瓶颈/容错 (15 分钟)**：挑 1-2 个核心难点深入（通常是并发一致性或热点），再谈扩展、缓存、限流、容灾。

【设计思想】“先做简单版本，再演进”

面试官最想看到的叙事是：**Start simple, then evolve**。先给一个能跑通的单体/单库方案，再主动指出它的瓶颈（“当 QPS 到 X，这里会成为热点”），然后引入缓存/分片/队列来解决。这种“演进式”叙事远胜于一上来就堆满 Kafka、Redis、分库分表却说不清为什么。复杂度要由问题逼出来，而不是为了炫技。

4.2 容量估算速查

【要点】常用数量级

1 天 $\approx 86,400$ 秒 $\approx 10^5$ 秒。所以“日 1 亿次请求” ≈ 1000 QPS (均摊)，峰值按 2-5 倍估。单条记录按 1 KB 估；1 亿条 ≈ 100 GB。内存读 $\sim$ 微秒级，SSD ~ 0.1 ms，磁盘 ~ 10 ms，跨地域网络 ~ 100 ms。

4.3 一致性、CAP 与权衡

分布式系统设计的取舍，绝大多数都能归结到“一致性 vs 可用性 vs 延迟”这条轴上。

- **CAP**：网络分区 (P) 不可避免，因此真正的取舍是分区时选 **C** (一致性) 还是 **A** (可用性)。订单/支付偏 C；信息流/搜索结果偏 A。
- **强一致 vs 最终一致**：强一致代价是延迟和可用性。什么时候必须强一致？——涉及钱、涉及“不能重复占用的资源”（如同一房源同一晚）。其余大多数场景，最终一致足矣。

【设计思想】一致性是“分区域”选择的，不是全局的

高手的设计不会笼统地说“整个系统强一致”。而是**按数据域分别选择**：预订的“占位”这一步必须强一致（防超卖）；而房源的浏览量、评分均值、搜索索引，最终一致即可。把强一致的范围**收缩到最小的关键路径**，是兼顾正确性与性能的核心手艺。

【追问】“如果这里数据不一致了会怎样？”

面试官常用这个问题探你对一致性的理解深度。应答框架：先说明这个数据域选了哪种一致性及其原因，再说不一致的具体后果（用户看到旧评分 vs 一个房源被两人订走），最后给兜底手段（对账任务、补偿事务、唯一约束作为最后防线）。

4.4 画架构图的小建议

白板/共享文档上，用清晰的方框 + 箭头即可。建议固定从左到右的层次：Client -> CDN/Gateway -> Service -> Cache -> DB，旁路挂 MQ、Search、Object Store。后面几章的架构图都用 ASCII 框图给出，你可以照着练手绘。

Chapter 5

设计订房系统

订房 (Booking) 是 Airbnb 业务的心脏，也是系统设计面试里出现频率最高的一题。它的迷人之处在于：表面上只是“点一下按钮把房子订下来”，但只要面试官追问一句“如果两个人同时订同一间房的同一晚怎么办”，整道题就立刻从 CRUD 升级成一道关于**并发控制、一致性与分布式事务**的硬核设计题。

本章严格按照系统设计答题五步法展开：需求澄清、容量估算、高层架构、数据模型与 API、深挖核心难点。其中第五步——**如何防止同一房源同一日期段被重复预订（超卖）**——是整章的灵魂，我们会把三种并发方案掰开揉碎对比。

【设计思想】先做简单版本，再演进

系统设计面试不是比谁一上来就掏出最复杂的架构。正确的节奏是：先澄清需求、画一个能跑通的最小可用版本（单库单表 + 事务），再由面试官的追问驱动你逐步演进（加缓存、分库分表、引入消息队列、做异地多活）。一上来就讲 Redlock、Saga、CRDT，反而暴露你“为了炫技而设计”。**所有的复杂度都应该是被某个具体需求逼出来的**，能说清“为什么需要”比“知道有这个东西”重要得多。

5.1 第一步：需求澄清

不要急着画框图。先用两三分钟和面试官对齐边界，这一步本身就是考点。

5.1.1 功能性需求

- 房客可以查询某房源在某日期段 (check-in / check-out) 是否可订。
- 房客可以发起预订，系统锁定该日期段，进入待支付状态。
- 房客完成支付后，预订确认；超时未支付则自动释放库存。
- 房客与房东都可以取消预订（触发退款与库存释放）。
- 同一房源、同一晚**绝不能**被两个有效订单同时占用（核心约束）。

5.1.2 非功能性需求

- **强一致性优先于可用性**：宁可让用户重试，也绝不能超卖。订房是“钱货两讫”的强一致场景，这与社交 feed 那种“最终一致就行”的场景有本质区别。
- **高可用**：搜索与浏览路径要 4 个 9；下单路径允许略低，但要快速失败而非阻塞。
- **延迟**：可用性查询 $p95 < 200\text{ms}$ ；下单 $p99 < 1\text{s}$ 。
- **幂等**：用户网络抖动重复点击“预订”，不能产生两笔订单。

【要点】一致性定调

订房系统的库存写路径是**强一致 (CP)** 场景：在 CAP 三选二里，当网络分区发生时，我们选择牺牲部分可用性（拒绝/重试），也要保证不超卖。但读路径（搜索、房源详情）可以是**最终一致 (AP)**，容忍秒级延迟。把同一个系统拆成“强一致的写核心”与“最终一致的读外围”，是本章最重要的架构直觉。

5.2 第二步：容量估算

估算的目的不是算出精确数字，而是判断**瓶颈在哪、要不要分片、热点有多热**。给出假设、推导数量级即可。

【要点】容量估算（数量级）

假设：

- 平台活跃房源 $\approx 6 \times 10^6$ (600 万)。
- 日活跃用户 DAU $\approx 1 \times 10^8$ (1 亿)，其中真正下单的转化率很低。
- 每天成功预订 $\approx 2 \times 10^6$ (200 万单)。

推导：

$$\text{写 QPS}_{avg} = \frac{2 \times 10^6}{86400} \approx 23 \text{ 单/秒}$$

写入并不大，但**峰值**可达均值的 10-20 倍（约 300-500 QPS），且高度集中在热门房源 / 节假日。

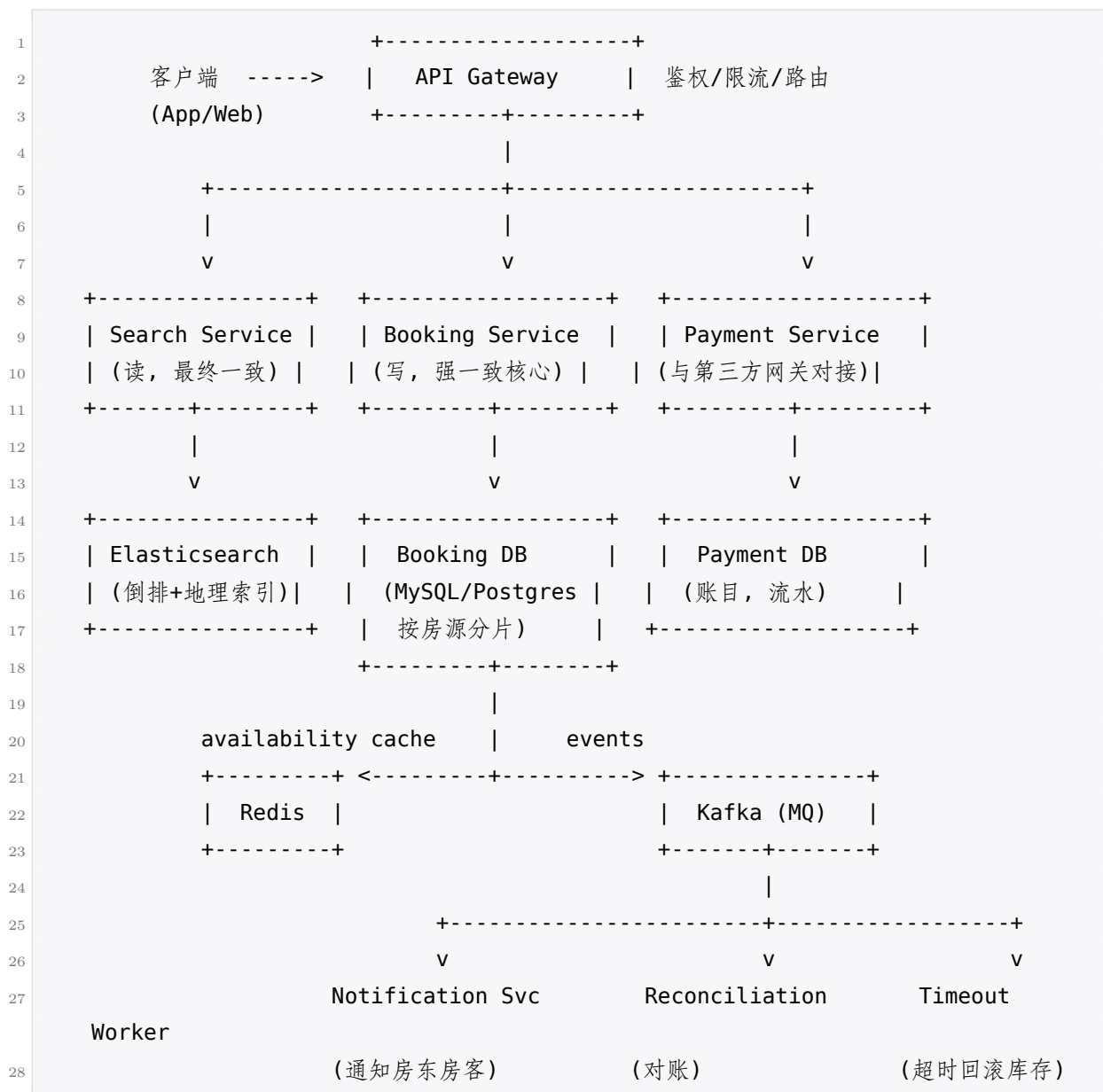
$$\text{可用性查询 QPS}_{avg} = \frac{1 \times 10^8 \times 5}{86400} \approx 5800/\text{秒}$$

（假设每个 DAU 平均看 5 次日历），峰值可破 10 万 QPS。存储：每单约 1KB，一年 $\approx 7.3 \times 10^8$ 单 $\times$ 1KB $\approx$ 730 GB/年，可控；但日历可用性数据（房源 $\times$ 未来一年每一天）规模更大，是缓存重点。

结论：**写不大但要强一致且有热点；读极大且可缓存**。这直接决定了后面的架构——库存写走关系型数据库 + 行级锁，读走 Redis 缓存。

5.3 第三步：高层架构

先画一个清晰的最小架构，再标注每个组件的职责。优先用 ASCII 框图，稳妥且符合本书风格。



关键设计选择：

- **库存（库存就是“日历可用性”）放在关系型数据库**，因为它需要事务和约束来保证不超卖。NoSQL 也能做，但要自己实现条件更新，关系库开箱即用。
- **支付服务独立**，与订房解耦。订房只负责“锁库存 + 建订单（PENDING）”，支付成功后通过事件回调把订单推进到 CONFIRMED。
- **Kafka 承载异步副作用**：发通知、对账、超时回滚都不该卡在下单主链路上。
- **Timeout Worker 消费“订单创建”事件**，到期检查未支付则补偿释放库存。

【设计思想】解耦：把副作用赶出主链路

下单主链路只做**必须强一致、必须同步完成**的两件事：扣库存、建订单。发邮件、发推送、更新搜索索引、记录埋点、对账——这些都是“副作用”，应该通过消息队列异步化。这样做有三个好处：主链路更快（p99 更低）、主链路更稳（下游挂了不影响下单）、各消费者可独立扩容与重试。这条“同步做核心、异步做外围”的分界线，几乎适用于所有写密集系统。

5.4 第四步：数据模型与核心 API

5.4.1 数据模型

核心是把“库存”建模成一张可以加约束的表。有两种主流建模方式：

方式 A：按“房源 + 日期”一行一天（推荐用于强约束）

```

1  -- 每个房源的每一天一行；可订与否一目了然
2  CREATE TABLE room_availability (
3      listing_id    BIGINT      NOT NULL,
4      date          DATE        NOT NULL,
5      status        TINYINT     NOT NULL,    -- 0=可订, 1=已占用, 2=房东关闭
6      booking_id    BIGINT      NULL,        -- 占用它的订单
7      version       INT         NOT NULL DEFAULT 0, -- 乐观锁版本号
8      PRIMARY KEY (listing_id, date)        -- 天然防止同一天两行
9  );
10
11 CREATE TABLE bookings (
12     booking_id    BIGINT      PRIMARY KEY,
13     listing_id    BIGINT      NOT NULL,
14     guest_id      BIGINT      NOT NULL,
15     check_in      DATE        NOT NULL,
16     check_out     DATE        NOT NULL,    -- 半开区间 [check_in, check_out)
17     status        VARCHAR(16) NOT NULL,    -- PENDING/CONFIRMED/CANCELLED/EXPIRED
18     idempotency_key VARCHAR(64) UNIQUE,    -- 幂等键，防重复下单
19     amount_cents  BIGINT      NOT NULL,
20     currency      CHAR(3)     NOT NULL,
21     created_at    TIMESTAMP   NOT NULL,
22     expires_at    TIMESTAMP   NULL        -- PENDING 的支付截止时刻
23 );

```

方式 A 的杀手锏：把多天预订拆成多行，配合主键 (listing_id, date) 与事务，超卖问题在数据库层就被天然约束住了。代价是：订一周要写 7 行，长租场景行数膨胀。

方式 B：一行一个预订，存日期区间（省空间，但要自己查重叠）

```

1  -- bookings 表直接存 [check_in, check_out) 区间
2  -- 下单前必须查：是否存在与新区间重叠的有效订单
3  -- 两个区间 [a1,a2) 与 [b1,b2) 重叠的充要条件：a1 < b2 AND b1 < a2
4  SELECT 1 FROM bookings

```

```

5 WHERE listing_id = :lid
6     AND status IN ('PENDING','CONFIRMED')
7     AND check_in < :new_check_out
8     AND :new_check_in < check_out
9 LIMIT 1;    -- 命中则冲突

```

【设计思想】区间重叠判定是可迁移的基本功

判断两个半开区间 $[a_1, a_2)$ 与 $[b_1, b_2)$ 是否重叠，记住一个对称式即可： $a_1 < b_2$ 且 $b_1 < a_2$ 。等价地，“不重叠”当且仅当一个区间整体在另一个左边或右边（ $a_2 \leq b_1$ 或 $b_2 \leq a_1$ ）。这个判定在日历、会议室预订、限流时间窗、基因组区间、CPU 调度里反复出现。务必用半开区间 $[in, out)$ 建模——check-out 当天房子是空的、可被下一位房客 check-in，半开区间让“连住交接”天然不冲突。

5.4.2 核心 API 契约

```

1 # 查询可用性（读，走缓存）
2 GET /v1/listings/{listingId}/availability?check_in=2026-07-01&check_out=2026-07-05
3 -> 200 { "available": true, "price_cents": 120000, "currency": "USD" }
4
5 # 发起预订（写，强一致）。客户端生成 Idempotency-Key
6 POST /v1/bookings
7 Header: Idempotency-Key: 8f3c...e91 # 同一 key 重复请求返回同一结果
8 Body: {
9     "listing_id": 998877,
10    "guest_id": 12345,
11    "check_in": "2026-07-01",
12    "check_out": "2026-07-05"
13 }
14 -> 201 { "booking_id": 55, "status": "PENDING", "expires_at": "...", "pay_url": "..."}
15 -> 409 { "error": "DATES_UNAVAILABLE" } # 已被占用，让前端刷新日历
16
17 # 确认支付（通常由支付回调驱动，也支持查询）
18 POST /v1/bookings/{bookingId}/confirm
19 -> 200 { "status": "CONFIRMED" }
20
21 # 取消
22 POST /v1/bookings/{bookingId}/cancel
23 -> 200 { "status": "CANCELLED", "refund": {...} }

```

【设计思想】幂等键：分布式系统的安全带

网络是不可靠的：客户端发了请求没收到响应，不知道服务端到底成没成，于是重试。如果“发起预订”不是幂等的，一次重试就可能变成两笔订单、两次扣款。解法是**幂等键(Idempotency-Key)**：客户端为每次“逻辑操作”生成一个唯一 key 放进请求头，服务端用它做唯一约束。第一次请求落库并记录 key 与结果；后续携带同一 key 的请求直接**返回首次的结果**，而不重复执行副作用。幂等是“至少一次投递”的世界里实现“恰好一次效果”的关键，它和重试、超时、消息队列消费是一套组合拳。

5.5 第五步（核心难点）：如何防止超卖

这是整道题的胜负手。同一房源、同一晚被两笔订单占用，就是“超卖”。我们对比三种并发控制方案，讲清各自的适用边界与坑。

5.5.1 方案 A：数据库事务 + 悲观锁 (SELECT ... FOR UPDATE)

思路：在一个事务里，先用 FOR UPDATE 锁住目标行，再检查并更新，最后提交。锁住期间别的事务只能等待。

```
1 BEGIN;
2 -- 锁住这次入住涉及的所有日期行（方式 A 建模）
3 SELECT status FROM room_availability
4 WHERE listing_id = 998877
5     AND date >= '2026-07-01' AND date < '2026-07-05'
6 FOR UPDATE;                                -- 悲观锁：其他事务在此阻塞
7
8 -- 应用层校验：若任意一天 status != 0 则冲突，回滚
9 UPDATE room_availability
10 SET status = 1, booking_id = 55
11 WHERE listing_id = 998877
12     AND date >= '2026-07-01' AND date < '2026-07-05'
13     AND status = 0;                        -- 再次确认仍可订
14
15 -- 受影响行数必须等于天数(4)，否则说明有人抢先，回滚
16 INSERT INTO bookings (...) VALUES (...);
17 COMMIT;
```

优点：语义最直观，强一致，数据库帮你保证原子性与隔离性。**缺点：**锁持有期间吞吐受限；热门房源会成为锁热点，事务排队；若事务里夹了慢操作（比如调用支付）会长时间持锁——**所以绝不能在持锁事务里做外部调用。**

【陷阱】在持锁事务里调用支付——经典死亡操作

新手最常见的错误：在 BEGIN ... COMMIT 之间，锁住库存后**同步调用第三方支付网关**，等支付成功再提交。支付可能耗时几秒甚至超时，这把数据库行锁持有了几秒，热门房源的并发

下单瞬间雪崩, 连接池被占满, 整个服务被一笔慢支付拖垮。正确做法: 事务只做“锁库存 + 建 PENDING 订单”然后**立刻提交**; 支付在事务**之外**异步进行, 成功后再用一个新事务把订单推进到 CONFIRMED。“数据库事务要短、要快、绝不跨网络”是铁律。

5.5.2 方案 B: 乐观并发控制 (version / CAS)

思路: 不提前加锁, 假设冲突很少。读出数据连同版本号, 更新时用 `WHERE version = :old_version` 做条件更新 (CAS); 若受影响行数为 0, 说明有人抢先改过, 本次失败、让上层重试。

```

1  -- 读: 拿到当前 status 与 version
2  SELECT status, version FROM room_availability
3  WHERE listing_id = 998877 AND date = '2026-07-01';    -- 设 version=7
4
5  -- 写: 条件更新, 只有 version 仍是 7 才成功 (Compare-And-Swap)
6  UPDATE room_availability
7      SET status = 1, booking_id = 55, version = version + 1
8      WHERE listing_id = 998877 AND date = '2026-07-01'
9          AND status = 0 AND version = 7;
10 -- affected_rows == 1 -> 抢到; == 0 -> 有人抢先, 失败重试

```

优点: 无锁、吞吐高, 适合**冲突概率低**的场景 (绝大多数房源的绝大多数日期并不抢手)。**缺点:** 高冲突 (爆款房源、秒杀式抢订) 下重试风暴严重, 性能反而不如悲观锁。多天预订要保证全部成功否则回滚, 逻辑更复杂。

【设计思想】乐观锁 vs 悲观锁: 赌的是“冲突率”

两者的本质区别是对冲突概率的**下注方向**。悲观锁假设“冲突很常见”, 所以先上锁、宁可让人等; 乐观锁假设“冲突很罕见”, 所以先干活、出事了再重试。选择标准很简单: **读多写少、冲突稀疏 -> 乐观锁** (重试代价低, 省掉了加锁开销); **写多、热点集中、冲突频繁 -> 悲观锁** (重试代价高, 不如老老实实排队)。这个判断框架可以原样迁移到库存、账户余额、计数器、文档协同编辑等所有并发更新场景。Airbnb 大多数房源冷门, 乐观锁是不错的默认值; 对极少数爆款房源可降级为悲观锁或排队。

5.5.3 方案 C: Redis 分布式锁——能用, 但别当成银弹

思路: 下单前先用 `SET key value NX PX 30000` 抢一把分布式锁 (key 形如 `lock:listing:998877:2026-07-01`) 抢到才进 DB 操作, 完事释放。它能在**请求到达数据库之前**就挡掉大部分并发, 给 DB 减压。

```

1  -- 加锁: NX=不存在才设置, PX=30s 自动过期(防死锁), value=唯一token
2  SET lock:listing:998877:2026-07-01 <token> NX PX 30000
3
4  -- 释放: 必须用 Lua 保证"判断token归属"与"删除"原子, 防误删他人锁
5  EVAL "if redis.call('get',KEYS[1])==ARGV[1]
6        then return redis.call('del',KEYS[1]) else return 0 end"
7        1 lock:listing:998877:2026-07-01 <token>

```


【陷阱】Redis 锁的三个致命风险——别拿它替代 DB 约束

1. **锁过期 vs 业务未完成**: PX 设 30s, 但业务卡了 31s, 锁自动释放, 第二个人拿到锁, 两人同时操作——超卖。加“看门狗”续租能缓解, 但增加复杂度。
2. **Redlock 争议**: 单点 Redis 挂了锁就没了; 用多节点 Redlock 又被 Martin Kleppmann 质疑在时钟漂移、GC 停顿下并不安全。这是工业界至今没有定论的争论。
3. **它只是“优化”不是“保证”**: Redis 锁能减少冲突到达 DB, 但不能作为正确性的最后防线。释放锁的瞬间、网络分区、进程暂停都可能破坏互斥。

正确姿势: 把 Redis 锁当作**第一道减压阀** (挡掉 99% 的并发), 但**最终正确性必须由数据库的唯一约束 / 条件更新兜底**。即“Redis 抢锁是为了快, DB 约束是为了对”。永远要有一个不依赖时钟、不依赖锁的强一致兜底。

5.5.4 三方案对比与推荐

方案	一致性	吞吐	适用场景
悲观锁 FOR UPDATE	强	中 (锁排队)	高冲突热点; 逻辑要求绝对正确
乐观锁 version/CAS	强	高 (低冲突时)	读多写少、冲突稀疏的常态房源
Redis 分布式锁	弱 (需 DB 兜底)	很高	作为前置减压阀, 不可独立保证正确性

面试推荐答法: 默认用乐观锁 (方案 B) 应对常态低冲突, 配合 (listing_id, date) 主键 / 唯一约束做最终兜底; 对极少数爆款房源, 前置一层 Redis 减压 (方案 C) + 数据库条件更新; 当冲突确实剧烈时局部退化为悲观锁 (方案 A)。核心结论是一句话: **Redis 用来快, 数据库用来对, 二者职责分明**。

5.6 预订生命周期: 状态机与超时回滚

预订是一个有明确生命周期的实体, 用**状态机**建模能让流程清晰、不留中间态漏洞。



支付与预订解耦后，PENDING 是个危险的中间态：库存被锁住但钱还没到。如果用户关掉页面再也不回来，库存会被永久占用。解决方案是**超时回滚**：下单时写入 `expires_at`（如 15 分钟后），并发一条延时事件到 Kafka；Timeout Worker 到期检查——若仍是 PENDING，则用**补偿事务**把订单置为 EXPIRED 并释放库存。

【设计思想】Saga 与补偿事务：跨服务一致性的务实解法

“锁库存 -> 扣款 -> 确认订单”跨越订房、支付两个服务两个数据库，无法用单个 ACID 事务覆盖。强行上两阶段提交（2PC）会带来同步阻塞与协调者单点，工业界很少在高并发链路用。更务实的是 **Saga 模式**：把长事务拆成一串本地事务，每步都配一个**补偿动作**（反向操作）。任一步失败，就按相反顺序执行已完成步骤的补偿，把系统“扳回”一致状态。例如支付失败 -> 补偿“释放库存 + 订单置 EXPIRED”。Saga 放弃了过程中的强隔离（允许出现短暂中间态），换来了高可用与可扩展，是微服务跨服务一致性的主流答案。配合幂等键，补偿动作可安全重试。

【陷阱】超时释放与延迟到账的竞态

设想：订单 15 分钟超时，Timeout Worker 第 15 分 0 秒把它置为 EXPIRED、释放了库存；但支付回调在第 15 分 1 秒姗姗来迟，把订单确认了——结果是“库存已释放、订单却 CONFIRMED”，甚至别人已经订走了这几天，造成**超卖 + 一笔需要退款的幽灵订单**。防御要点：状态机转移必须是**带条件的原子更新**（`UPDATE ... WHERE status='PENDING'`），EXPIRED 之后任何确认都必须失败并触发自动退款；支付服务侧也要校验订单仍处于可确认状态。一切状态推进都要“先看当前态、再用 CAS 推进”，绝不无条件覆盖。

5.7 瓶颈、容错与扩展

5.7.1 分片 (Sharding)

按 `listing_id` 哈希分片，把不同房源的库存与订单打散到不同库。好处：同一房源的所有日期落在同一分片，下单事务是**单分片本地事务**，无需分布式事务。坏处：爆款房源会造成**分片热点**。缓解：对超热房源做单独隔离 / 排队，或把“房源 × 日期范围”二级打散。

5.7.2 缓存可用性，读写分离

可用性查询 QPS 是写的几百倍，且读多写少，用 Redis 缓存房源日历。**写路径成功扣库存后，主动失效或更新对应缓存**（write-through 或先改库再删缓存）。读走只读副本，进一步分担主库压力。

【设计思想】缓存失效：先改库，再删缓存

缓存与数据库的一致性是经典难题。推荐 **Cache-Aside + 先更新数据库、再删除缓存**（而非更新缓存）。删除而非更新，是因为“删”是幂等的、并发下不易产生脏值；下次读未命中自然回源重建。即便如此仍有极小概率的并发竞态（读旧值回填），可用**延迟双删**或给缓存设较短 TTL 兜底。记住：**缓存是性能优化，不是真相来源 (source of truth)**；数据库才是真

相。任何“缓存对了但数据库错了”的设计都是本末倒置。

5.7.3 限流与熔断

API Gateway 层做**限流**（令牌桶 / 漏桶）防止恶意刷单与突发洪峰压垮库存库；对支付等外部依赖做**熔断**（连续失败则快速失败并降级），避免级联雪崩。下单失败要给用户清晰反馈（“手慢了，请刷新日历”）而非长时间转圈。

5.7.4 容灾与多活

库存库做主从复制 + 自动故障转移。跨区域可做**异地多活**，但库存的强一致要求使跨区域写很棘手——常见折中是按房源地理位置就近**单元化（cell-based）**：某房源的库存只在其归属单元强一致写，跨单元只读复制。这把“全局强一致”降级为“单元内强一致 + 单元间最终一致”，是大规模系统的常见妥协。

【追问】追问：双十一级别的爆款房源，乐观锁重试风暴怎么办？

回答分层：(1) 承认乐观锁在极高冲突下会重试风暴，吞吐反降。(2) 第一招**排队削峰**：把对热点房源的下单请求送入一个串行队列（每房源一个 partition），由单消费者顺序处理，把并发冲突变成串行无冲突——本质是用排队消除竞争。(3) 第二招**热点隔离**：识别爆款房源并由到专用资源池 / 专库，避免拖累普通房源。(4) 退一步，对该房源局部改用悲观锁，让冲突在 DB 内排队而非应用层空转重试。(5) 产品手段：库存预扣 + 倒计时支付，提前过滤无效流量。关键是表达“我知道乐观锁不是万能的，会按冲突强度切换策略”。

【追问】追问：如何保证支付与订单状态最终一致？万一支付成功但确认订单的写失败了？

这是 Saga 的核心容错点。要点：(1) 支付服务以**支付流水**为权威记录，订房服务以**订单状态**为权威记录，两者通过事件/消息驱动同步。(2) “确认订单”这一步必须**幂等且可重试**——支付成功事件投递到 Kafka，订房消费者消费时用幂等键保证重复消费只生效一次；若确认写失败，消息不 ack，会被**重新投递**直到成功。(3) 引入**对账（reconciliation）**定时任务：扫描“支付成功但订单未确认”或“订单确认但无支付”的不一致记录，自动补偿或人工介入。(4) 用户侧看到的是“支付处理中”，确认完成后再通知。本质是：**异步消息保证最终一致 + 对账兜底 + 幂等消费**三件套，这是所有涉及钱的系统的标准配置。

【要点】本章速记

- 写强一致（库存不超卖）、读最终一致（可用性可缓存），一个系统两种一致性。
- 防超卖三方案：悲观锁（高冲突）、乐观锁 CAS（低冲突，默认）、Redis 锁（前置减压，不可独立保证正确）。**Redis 求快，DB 约束求对**。
- 用半开区间 $[in, out)$ 建模日期，重叠判定 $a_1 < b_2 \wedge b_1 < a_2$ 。
- 幂等键防重复下单；事务要短、绝不在持锁事务里调外部服务。
- 支付与订房解耦，用状态机 + Saga 补偿 + 超时回滚 + 对账保证最终一致。

Chapter 6

设计房源搜索

如果说订房系统考的是**写一致性**，那么房源搜索考的就是**读规模**。用户在地图上拖动、输入“7月1日到5日、巴黎、2居室、可带宠物、预算每晚200美元以内”，期望几百毫秒内返回一屏既相关又当下可订的房源。这背后糅合了**地理检索、多维过滤、可用性约束、相关性排序、分层缓存**五个难点，是 Airbnb 系统设计面试的另一道经典大题。

本章同样按五步法展开，重点讲清两件事：**地理空间索引怎么选**(GeoHash / S2 / QuadTree)，以及“**日期可用性**”这种高频变化的维度如何与搜索引擎结合。

【设计思想】读多写少系统的设计基调

搜索是典型的**读远大于写**系统：房源信息一天可能才更新几次，但每秒有上万次查询。这类系统的设计基调与订房恰好相反——可以**大胆牺牲强一致换取读性能**：用倒排索引、用多级缓存、用只读副本、容忍索引比数据库晚几秒（最终一致）。记住这条分界：**钱和库存要强一致，浏览和搜索可最终一致**。把系统按“读外围 / 写核心”切开，分别采用 AP 与 CP 策略，是贯穿本书的架构主线。

6.1 第一步：需求澄清

6.1.1 功能性需求

- 地理检索：给定地图视野（中心点 + 半径，或一个矩形边界框 bounding box）返回范围内房源。
- 多维过滤：日期可用性、价格区间、房型（整租/合租）、卧室数、设施（WiFi/泳池/宠物）、评分等。
- 排序与个性化：按相关性、价格、距离、评分排序，并对用户做个性化打分。
- 分页：稳定、无重复、无遗漏地翻页。

6.1.2 非功能性需求

- **低延迟**：搜索 p95 < 300ms，地图拖动要近实时。
- **高可用**：搜索是流量入口，要 4 个 9；宁可返回稍旧的数据，也不要返回错误。

- **最终一致**：房源新增 / 改价 / 可用性变化，容许秒级延迟体现在搜索结果里。
- **高吞吐**：峰值读 QPS 十万级。

6.2 第二步：容量估算

【要点】容量估算（数量级）

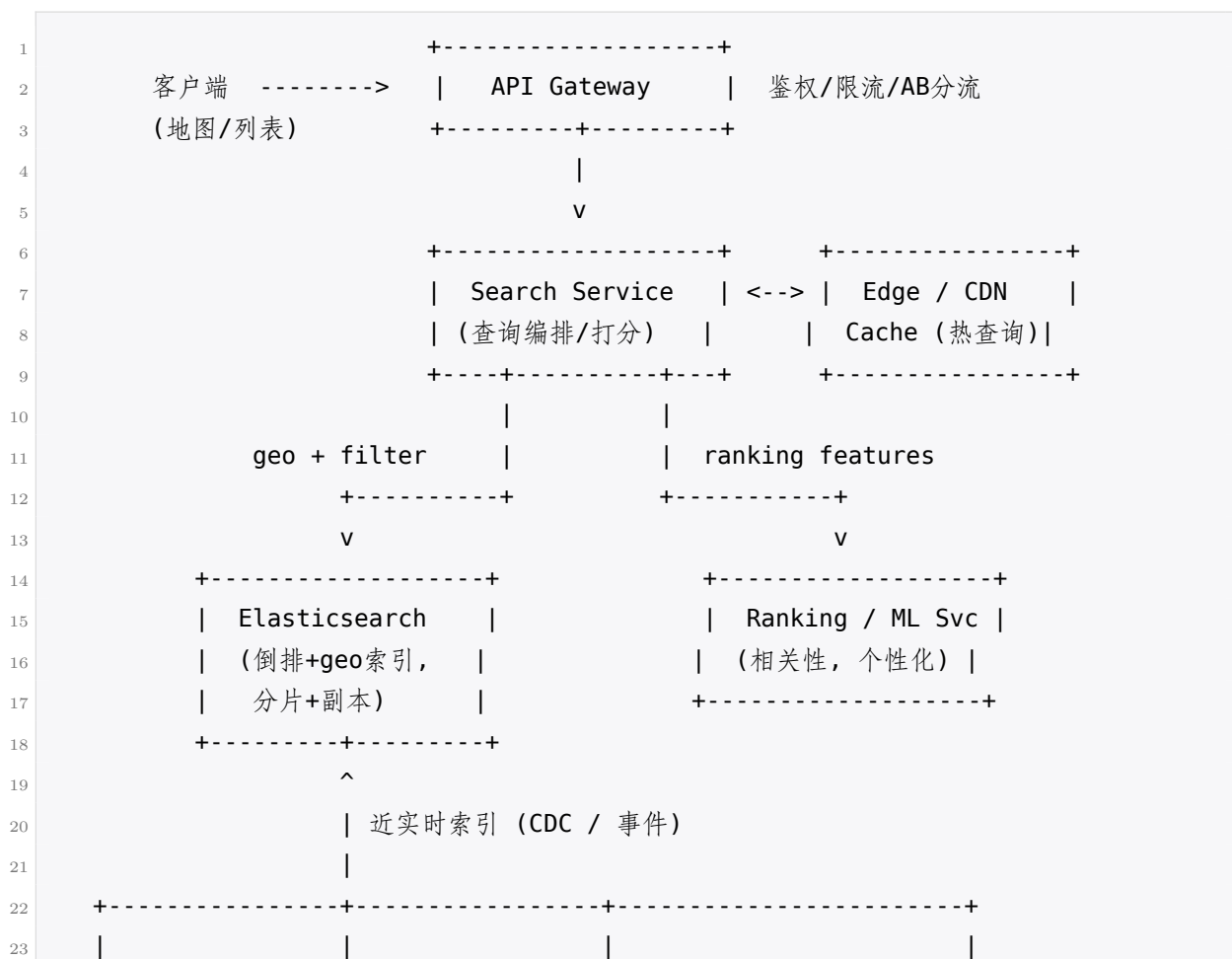
假设：活跃房源 $\approx 6 \times 10^6$ ；DAU $\approx 1 \times 10^8$ ；人均每天 10 次搜索。

$$\text{搜索 QPS}_{avg} = \frac{1 \times 10^8 \times 10}{86400} \approx 1.16 \times 10^4 / \text{秒}$$

峰值按均值 8-10 倍计 $\approx 10^5$ QPS。索引数据量：每房源索引文档约 2KB（含地理坐标、设施、价格、文本）， $6 \times 10^6 \times 2\text{KB} \approx 12\text{ GB}$ ——**完全可以放进内存 / 分布式倒排索引**，这决定了我们能用 Elasticsearch 这类内存型搜索引擎。可用性数据： 6×10^6 房源 $\times$ 未来 365 天 $\approx 2.2 \times 10^9$ 个“房源-日”状态位，是搜索预过滤的难点（后面详述）。

结论：索引数据可全量入内存，瓶颈不在存储而在“高 QPS 的复杂多维查询 + 高频变化的可用性维度”。

6.3 第三步：高层架构



24	+-----+	+-----+	+-----+	+-----+
25	Listing	Availability	Pricing Svc	Kafka (CDC)
26	DB	Service/DB		----> binlog 流
27	+-----+	+-----+	+-----+	+-----+
28	(真相来源)	(库存/日历)	(动态定价)	

关键设计选择：

- **数据库是真相来源 (source of truth)**，搜索引擎是派生的读视图。房源写入 Listing DB，通过**变更数据捕获 (CDC)** 把 binlog 推到 Kafka，再由索引管线近实时更新 Elasticsearch。
- **Search Service 做编排**：解析查询、调 ES 拿候选集、调 Ranking 服务重排、组装结果、读缓存。
- **多级缓存**：CDN/边缘缓存热门城市的“无个性化”结果，应用层缓存 geo 网格结果。

【设计思想】CQRS：把“写模型”和“读模型”分开

Listing DB 为了写得规范，是高度规范化 (normalized) 的关系表；但搜索需要的是**反规范化 (denormalized)**、**为查询优化的宽文档**。与其在一个模型里两头将就，不如**读写分离到不同模型**——这就是 CQRS (命令查询职责分离)。写走 DB，读走 ES；二者用事件管线 (CDC + Kafka) 异步同步，天然是最最终一致。这个模式的可迁移价值在于：**当读和写的访问模式差异巨大时，不要硬塞进同一个存储，用派生视图各自优化**。代价是要接受同步延迟与一份冗余数据，但对读密集系统这笔买卖非常划算。

6.4 第四步：数据模型与核心 API

6.4.1 搜索文档 (Elasticsearch)

```

1  {
2    "listing_id": 998877,
3    "title": "Cozy loft near Eiffel Tower",
4    "location": { "lat": 48.8584, "lon": 2.2945 },    // geo_point 类型
5    "s2_cells": ["47a1b2", "47a1b3"],              // 预计算的 S2 cell IDs
6    "city": "Paris", "country": "FR",
7    "room_type": "entire_home",
8    "bedrooms": 2,
9    "amenities": ["wifi", "pets_allowed", "pool"], // 倒排过滤
10   "price_per_night_cents": 18000,
11   "rating": 4.87, "review_count": 213,
12   "instant_book": true,
13   "updated_at": 1719734400
14 }

```

1 # 核心搜索 API

```

2 GET /v1/search?
3   bbox=48.80,2.25,48.90,2.36           # 地图边界框 (minLat,minLon,maxLat,maxLon)
4   &check_in=2026-07-01&check_out=2026-07-05
5   &guests=2&min_price=100&max_price=250
6   &room_type=entire_home&amenities=wifi,pets_allowed
7   &sort=relevance&page_token=eyJ...   # 游标分页 token
8 ->
9 200 {
10  "results": [ { "listing_id": 998877, "price_cents": 18000, ... }, ... ],
11  "next_page_token": "eyJ...",          # 不用 offset, 用游标
12  "total_approx": 1240                  # 近似总数即可
13 }

```

6.5 第五步（核心难点一）：地理空间索引

“找出地图框内的房源”看似简单，但二维范围查询用普通 B-Tree 索引会很糟（经纬度两个维度无法同时用一个有序索引高效裁剪）。专门的地理索引把二维空间映射成一维有序键，从而能用前缀 / 范围查询。三种主流方案：

6.5.1 GeoHash：把经纬度交错编码成字符串前缀

GeoHash 把地球递归二分，将 (lat, lon) 交错编码成一个 Base32 字符串。**前缀越长，网格越小、越精确；共享前缀越长的两点，地理上越接近。**查询时把目标区域转成若干 GeoHash 前缀，做前缀匹配即可。

- **优点：**实现简单、可读、天然支持前缀范围查询，几乎所有数据库都能存字符串前缀。
- **缺点（关键）：边界问题**——两个物理上紧邻的点，可能恰好落在网格边界两侧，前缀完全不同（比如赤道、本初子午线附近，或仅仅是网格分界线两侧）。所以查询必须**同时检索目标网格 + 周围 8 个邻居网格**，否则会漏掉近在咫尺却跨格的房源。此外 GeoHash 的网格在高纬度会变形（经线收敛）。

6.5.2 S2：把球面映射到立方体的 Hilbert 曲线

Google S2 把地球当作真正的球面，投影到外接立方体的 6 个面，再在每个面上用**希尔伯特曲线 (Hilbert curve)** 把二维 cell 编号成一维 64 位整数。

- **优点：**直接在球面上建模，**没有 GeoHash 那种经纬度矩形在两极严重变形的问题**，全球 cell 面积更均匀；希尔伯特曲线保证“一维上相邻的 cell，空间上也大概率相邻”，局部性极好；支持多分辨率 cell (cell level)，能用一组不同大小的 cell 精确覆盖任意圆/多边形区域；64 位整数比字符串更省更快。
- **缺点：**实现与心智模型更复杂，覆盖区域要算 cell covering。

- **为什么 S2 适合球面**：因为它从一开始就在球面/立方体上做等积性更好的划分，而 GeoHash 本质是在“经纬度平面矩形”上切分，纬度越高越失真。这正是 Uber、Google 等用 S2 的原因。

6.5.3 QuadTree：按密度自适应细分的四叉树

QuadTree 把空间递归地分成四个象限，哪里点密就在哪里继续细分，是一棵深度不均的树。

- **优点：自适应密度**——曼哈顿这种房源稠密区自动分得很细，撒哈拉沙漠这种稀疏区保持粗粒度，查询时每个叶子节点的候选数较均衡。
- **缺点**：通常是**内存中的树结构**，需要构建与维护、再平衡，分布式持久化不如前两者“一维 key”那么自然；动态插入删除有成本。

【设计思想】空间降维：把二维问题变成一维有序键

GeoHash 和 S2 的共同精髓是**用空间填充曲线 (space-filling curve) 把二维坐标降成一维有序键**，从而复用我们最熟悉的一维有序结构 (B-Tree、排序、范围扫描、前缀匹配)。这是一个极其通用的降维思想：高维 / 空间 / 多属性的“邻近查询”，常常可以通过某种保局部性 (locality-preserving) 的编码，转化为一维上的范围查询。理解了这一点，GeoHash 的前缀、S2 的 Hilbert 编号就不再是魔法，而是同一思想的不同实现。面试里能讲清“为什么要降维、降维保住了什么性质 (局部性)、丢了什么 (边界精度)”，远胜于背诵 API。

【陷阱】GeoHash 边界遗漏：只查中心格会漏房源

最常见的地理检索 bug：用户在地图上的位置恰好靠近某个 GeoHash 网格的边界，你只检索了它所在的那个网格，结果把**紧邻边界另一侧、明明很近的房源漏掉了**。正确做法是检索**中心格 + 周围 8 个邻居格** (共 9 格) 再做精确距离过滤；或者干脆用 S2 的 cell covering 算法生成能完整覆盖查询圆的一组 cell。另一个相关陷阱：直接用经纬度差的欧氏距离当真实距离——地球是球面，应使用 **Haversine (半正矢)** 公式算大圆距离，否则高纬度误差很大。

方案选择小结：

方案	键类型	适用与取舍
GeoHash	字符串前缀	简单通用，但有边界问题、高纬变形
S2	64 位整数 cell	球面均匀、局部性好，最适合全球尺度，复杂度高
QuadTree	内存树	自适应密度，适合分布不均，分布式持久化较难

面试可答：全球尺度、追求精度与均匀性选 S2；快速落地、已有支持 GeoHash 的存储选 GeoHash (注意补邻居格)；单机内存、密度极不均选 QuadTree。Elasticsearch 内置 geo_point + BKD-tree 也能直接做 bounding box / 距离查询，工程上常直接用它。

6.6 核心难点二：多维过滤与倒排索引

设施、房型、城市这类**离散属性**用**倒排索引 (inverted index)**：为每个取值维护一个“包含它的房源 ID 列表 (posting list)”，多个过滤条件就是对多个 posting list 求**交集**。价格、卧室数这

类数值范围用 ES 的 `range` 查询（底层 BKD-tree）。这正是 Elasticsearch / Lucene 的拿手好戏，所以工程上直接用 ES 承载 “geo + 多维过滤 + 文本相关性” 一站式查询。

```
1 # Elasticsearch 查询骨架: filter(不打分,可缓存) + must(打分)
2 {
3   "query": { "bool": {
4     "filter": [                                # 布尔过滤, 结果可缓存
5       { "geo_bounding_box": { "location": {...} } },
6       { "terms": { "amenities": ["wifi","pets_allowed"] } },
7       { "range": { "price_per_night_cents": { "gte":10000, "lte":25000 } } },
8       { "term": { "room_type": "entire_home" } }
9     ],
10    "must": [                                    # 参与相关性打分
11      { "match": { "title": "eiffel loft" } }
12    ]
13  }}
14 }
```

【设计思想】过滤与打分分离: filter 可缓存, query 才打分

搜索引擎里有个值得迁移的设计: 把“硬条件过滤”与“相关性打分”分开。过滤 (filter context) 只回答“符不符合”——是布尔结果, 与查询词无关、可被缓存复用 (同样的“巴黎 + WiFi + 整租”过滤位图能服务无数次不同关键词的搜索)。打分 (query context) 才计算“有多相关”, 是连续分数、随查询变化。这种“先用廉价的、可缓存的布尔过滤大幅缩小候选集, 再对小候选集做昂贵的打分/排序”的两阶段范式, 在搜索、推荐、广告检索里无处不在——便宜的粗筛在前, 昂贵的精排在后。

6.7 核心难点三: 日期可用性如何融入搜索

这是 Airbnb 搜索最棘手的部分: 用户几乎总会带日期, 而可用性是高频变化的 (每次成功预订都会改变某些房源某些日期的状态)。把每天的可用性实时同步进搜索索引代价高昂。两种策略:

6.7.1 预过滤 (pre-filter) vs 后过滤 (post-filter)

- **预过滤**: 把可用性也做进索引 (比如索引每个房源“已被占用的日期范围”或一个按天的 bitmap), 搜索时直接用日期条件过滤。优点: 结果集干净、分页准确; 缺点: 可用性变化要近实时回灌索引, 写放大严重, 索引更新压力大。
- **后过滤**: 搜索先忽略日期, 返回地理 + 属性匹配的候选集, 再调用 **Availability** 服务批量核验这批候选在目标日期是否可订, 过滤掉不可订的。优点: 索引无需承载高频可用性更新, 职责清晰; 缺点: 可能要多取一些候选以补偿被过滤掉的, 分页与总数变得不精确。

【设计思想】预过滤 vs 后过滤：在“索引新鲜度成本”与“查询召回成本”间权衡

这是一个普遍的检索权衡。**预过滤**把约束前置进索引，查询快但索引维护贵、新鲜度难保证；**后过滤**保持索引简单，把约束推迟到查询时核验，索引轻但查询要多捞候选、结果可能不足。选择取决于**该维度的变化频率与基数**：变化慢、基数低的维度（城市、房型）适合预过滤进索引；变化极快的维度（实时可用性、动态价格）更适合后过滤或近实时局部更新。实务中常**混合**：用粗粒度可用性（如“这个月是否还有空档”）做预过滤的廉价初筛，再用后过滤精确核验具体日期。**把易变的留到最后、把稳定的固化到索引**，是新鲜度与性能间的经典折中。

【陷阱】后过滤导致分页“缺斤短两”与重复

用后过滤时，ES 返回 20 条候选，经可用性核验后只剩 12 条可订——这一页就“缩水”了，若简单按 ES 的 offset 翻页，下一页还会因为重新过滤而**重复或遗漏**。两个要点：(1) 用**游标分页 (cursor / search_after)** 而非 offset 分页——offset 在数据变化时极易重复/跳漏，且深翻页 (offset 很大) 性能灾难；游标基于上一页最后一条的排序值定位，稳定且高效。(2) 后过滤时**超量召回 (over-fetch)**，比如要 20 条就先取 60 条候选) 以补偿过滤损耗，并把“总数”作为**近似值**呈现而非精确值。搜索结果的“大约 1200 间房源”本来就不需要精确。

6.8 核心难点四：排序、个性化与分层缓存

6.8.1 两阶段排序

先用 ES 的轻量打分 (BM25 文本相关性 + 简单 boost) 粗排出 top-N 候选，再把这 N 条送入 **Ranking/ML 服务** 做精排：综合相关性、价格竞争力、房东质量、历史转化、以及**个性化特征** (用户历史偏好、设备、地域)。这正是前述“便宜粗筛 + 昂贵精排”范式的落地。

6.8.2 分层缓存

- **边缘/CDN 缓存**：对无个性化、无日期的热门查询（“巴黎热门房源”）整页缓存，命中即返回，TTL 短（分钟级）。
- **应用层缓存**：缓存 geo 网格 + 常见过滤组合的候选 ID 列表。
- **个性化结果不缓存或按用户短缓存**：因为千人千面，缓存命中率低。

【陷阱】个性化与缓存的天然矛盾

新手容易想“把搜索结果都缓存起来就快了”，但**个性化和缓存是天生的敌人**：结果越个性化 (千人千面)，缓存键的基数越爆炸、命中率越低，缓存几乎失效。务实的分层策略是：**把可缓存的与不可缓存的拆开**——地理 + 属性的**过滤候选集**是用户无关的，可大力缓存；**个性化精排**放在候选集之上对小集合实时计算、不缓存或仅按用户做秒级短缓存。即“共享的粗筛结果重度缓存，私有的精排实时计算”。这条“分离可缓存层与个性化层”的思路，对推荐、信息流系统同样适用。

6.9 瓶颈、容错与扩展

6.9.1 ES 分片与副本

按地理（如国家/大区）或哈希对索引分片，每片多副本既扛读 QPS 又做容灾。地理分片的好处是“巴黎的搜索只打到欧洲分片”，但要警惕**热点城市分片过载**，必要时对超热城市单独拆片。

6.9.2 近实时索引管线

房源/价格/可用性的变更通过 CDC -> Kafka -> 索引 worker 写入 ES，达成**秒级最终一致**。要保证管线的**幂等**（同一变更重复消费不出错）与**有序**（同一房源的多次更新按序应用，可用版本号/时间戳做 last-write-wins）。

6.9.3 数据新鲜度的取舍

搜索结果允许**稍旧**：用户看到一间“几秒前刚被订走”的房源，点进去再提示不可订，是可以接受的体验（在下单环节由强一致的库存校验兜底）。**绝不能反过来**——为了搜索的新鲜度去拖累或强同步库存写路径。搜索的最终一致，正是靠订房环节的强一致来兜底。

【追问】追问：用户搜索时看到可订，点进去却已被订走，怎么办？

这是最终一致系统的必然现象，关键是**分层兜底而非追求搜索绝对准确**。(1) 坦诚承认：搜索索引是最终一致的，存在秒级窗口，强行让搜索实时准确代价过高且没必要。(2) **真正的正确性防线在下单环节**——用户点击预订时，由强一致的库存服务（见订房章）做权威校验，不可订就明确提示并刷新。(3) 体验优化：对**即时预订 (instant book)** 房源可在结果页做一次轻量可用性复核以减少“扑空”；可用性变化高频的房源可缩短其索引 TTL、提高回灌优先级。(4) 度量“搜索-下单转化中的扑空率”，超阈值才加强同步。核心表达：**把强一致放在唯一真正需要它的地方（下单），其余环节用最终一致换性能**。

【追问】追问：深翻页（看到第 100 页）为什么慢？怎么解决？

要点：(1) 解释 **offset 深翻页的代价**——`from=10000, size=20` 要求每个分片都取出前 10020 条再丢弃前 10000 条，协调节点还要归并，越往后越慢且耗内存，这是 ES（及大多数数据库 OFFSET）的通病。(2) 解决：用**游标分页 search_after**，基于上一页最后一条的排序值（如 `[score, listing_id]`）继续，跳过前面的扫描，复杂度与页码无关。(3) 产品上**限制最大可翻页数**——没人会真翻到第 500 页，搜索体验应靠“缩小过滤条件”而非无限翻页。(4) 游标分页还顺带解决了数据变动时 offset 分页的**重复/遗漏**问题。本质：用“记住上次位置”代替“数过前面多少个”。

【要点】本章速记

- 读多写少，基调是**最终一致换读性能**；DB 是真相、ES 是派生读视图（CQRS + CDC 同步）。
- 地理索引降维三选：GeoHash（简单、有边界问题）、S2（球面均匀、局部性好、全球首

选)、QuadTree (自适应密度)。空间填充曲线把二维变一维有序键。

- 多维过滤用倒排索引求交；filter 可缓存、query 才打分（粗筛在前、精排在后）。
- 可用性融合：预过滤（索引新鲜度贵）vs 后过滤（查询多召回），按维度变化频率权衡，常混合。
- 用游标分页防深翻页慢与重复；分离“可缓存粗筛”与“个性化精排”；搜索的最终一致由下单的强一致兜底。

Chapter 7

设计消息与支付系统

本章把两道高频系统设计题合在一起：**消息/通知系统与支付/信任系统**。它们看似无关，却共享同一组分布式心法——**幂等消费、最终一致、消息驱动解耦、不丢不重**。前者考的是“海量异步消息如何可靠扇出且不丢不重”，后者考的是“涉及真金白银时如何做到绝对正确且抗欺诈”。Airbnb 作为**双边市场 (two-sided marketplace)**，对支付与信任安全 (trust & safety) 的重视到了近乎偏执的程度，这一点要在答题中反复体现。

7.1 第一部分：消息与通知系统

7.1.1 第一步：需求澄清

功能性：

- 房东与房客之间的**实时一对一消息**（预订前咨询、入住沟通）。
- **系统通知**：预订确认、支付成功、入住提醒等（一对多扇出）。
- 多端同步、离线消息存储、消息历史拉取。
- 已读/未读状态、送达回执。

非功能性：

- **不丢消息**（at-least-once 投递）+ **不重复展示**（消费端幂等去重）。
- 实时性：在线用户端到端延迟 $< 1s$ 。
- 高可用、可水平扩展到亿级用户。
- 消息有序：同一会话内消息按发送顺序展示。

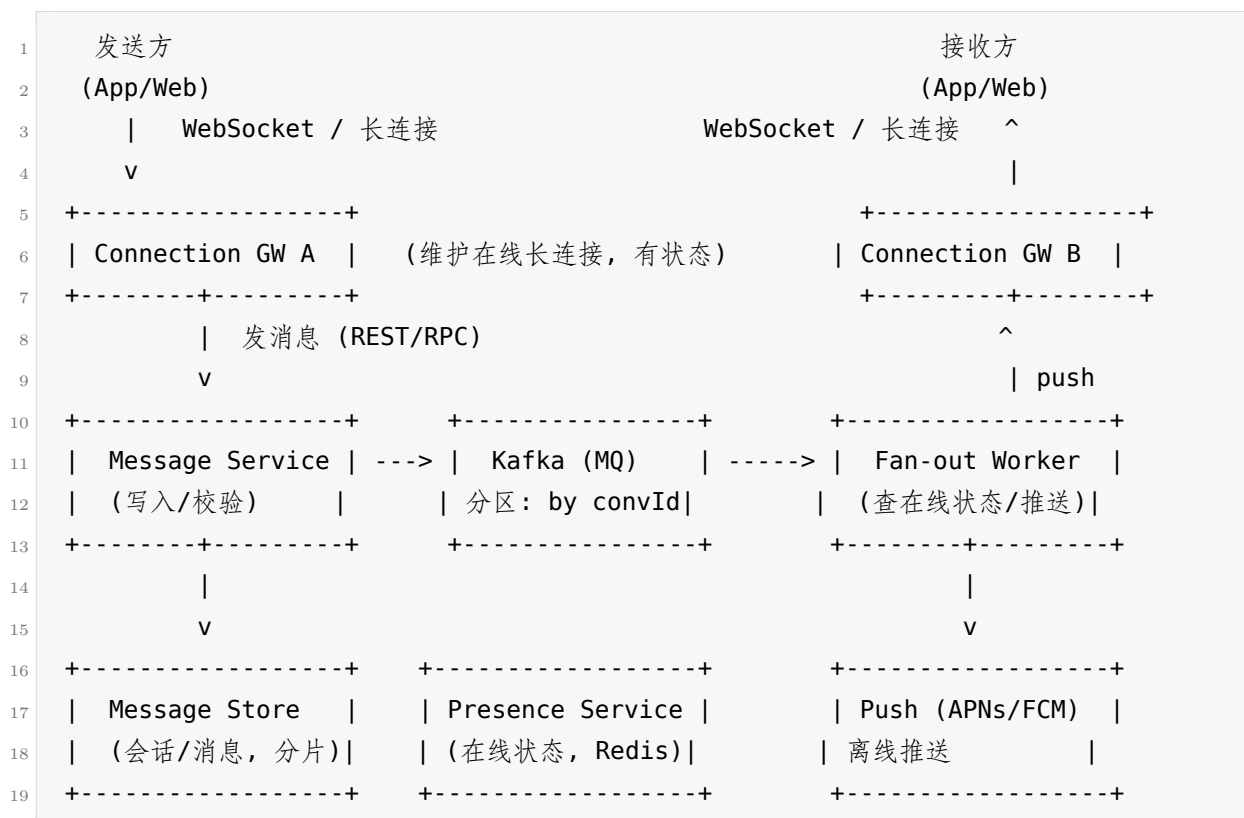
【要点】容量估算（数量级）

假设 $DAU \approx 1 \times 10^8$ ，人均每天收发 20 条消息（含系统通知）：

$$\text{消息写 QPS}_{avg} = \frac{1 \times 10^8 \times 20}{86400} \approx 2.3 \times 10^4 / \text{秒}$$

峰值约 2×10^5 /秒。每条消息约 1KB，日增 $\approx 2 \times 10^9 \times 1\text{KB} = 2 \text{ TB/天}$ ——消息存储要可**水平扩展、冷热分层**（近期热数据快存，历史冷数据归档）。在线长连接数：若 10% DAU 同时在线 = 10^7 条长连接，需要一层可水平扩展的**连接网关**。

7.1.2 第二步：高层架构



要点：连接网关（Connection Gateway）维护海量 WebSocket 长连接，是有状态组件；Message Service 负责落库与校验；Kafka 做异步扇出与削峰；Presence 服务记录谁在线（决定走实时推送还是离线推送）。

7.1.3 第三步：推还是拉？

【设计思想】 推拉结合：在线推、离线拉、历史拉

消息送达有两种基本范式。**推 (push)**: 服务端主动经长连接把消息推给在线用户, 实时性好, 但要维护长连接、用户离线时推送会浪费。**拉 (pull)**: 客户端定时来问 “有没有新消息”, 实现简单、对服务端无状态友好, 但实时性差、空轮询浪费。生产系统几乎都用**推拉结合**: 用户**在线时用推** (长连接实时下发); **离线时落库 + 走 APNs/FCM 离线推送**; **重新上线时用拉** (按 last_seen 游标拉取离线期间堆积的消息历史)。这个 “实时增量用推、批量补齐用拉” 的组合, 同样适用于 feed 流、通知中心、协同编辑。别在 “纯推” 或 “纯拉” 里钻牛角尖, 混合才是工程答案。

7.1.4 第四步：数据模型

```

1  -- 会话表（一对一/群）
2  CREATE TABLE conversations (
3      conv_id      BIGINT PRIMARY KEY,
4      type         TINYINT,           -- 0=一对一，1=系统通知
5      participants JSON,             -- 参与者列表
6      updated_at   TIMESTAMP
7  );
8
9  -- 消息表：按 conv_id 分片，会话内按 seq 严格有序
10 CREATE TABLE messages (
11     conv_id      BIGINT NOT NULL,
12     seq          BIGINT NOT NULL,    -- 会话内单调递增序号(保证有序)
13     message_id   BIGINT NOT NULL,    -- 全局唯一(雪花ID)，幂等去重用
14     sender_id    BIGINT NOT NULL,
15     body         TEXT,
16     created_at   TIMESTAMP,
17     PRIMARY KEY (conv_id, seq)
18 );
19
20 -- 每个用户在每个会话的已读位置(已读/未读由此推导)
21 CREATE TABLE read_cursors (
22     user_id      BIGINT,
23     conv_id      BIGINT,
24     last_read_seq BIGINT,           -- 已读到的 seq
25     PRIMARY KEY (user_id, conv_id)
26 );

```

【设计思想】已读未读：别为每条消息存一个标志位，存一个游标

“已读/未读”最朴素的实现是给每条消息每个接收者存一个布尔位——但这会产生海量小写、空间和写压力都爆炸。更优雅的做法是：每个用户在每个会话只存一个 **last_read_seq**（已读到的序号）。某条消息是否已读，由“它的 seq 是否 $\leq$ last_read_seq”推导；未读数 = 会话最新 seq - last_read_seq。这把“每消息每人一条状态”压缩成“每会话每人一个游标”，写入量从 $O(\text{消息数})$ 降到 $O(1)$ 。用一个单调游标代替海量离散状态位，是处理“进度/水位”类问题的通用技巧（也用于消费位点、播放进度、同步位点）。

7.1.5 第五步：不丢、不重、有序

【设计思想】at-least-once + 幂等消费 = 恰好一次效果

分布式消息几乎不可能做到传输层“恰好一次 (exactly-once)”投递——网络会重传，确认会丢失。工业界的标准答案是：**投递保证 at-least-once**（至少一次，宁可重复也不丢），再在消费端用幂等去重把“至少一次”变成“恰好一次的效果”。具体做法：每条消息带全局唯一

`message_id`，消费方维护一个“已处理 ID 集合”（或在落库时对 `message_id` 加唯一约束），重复到来的同一 ID 直接跳过。这套“可靠投递靠重试、最终正确靠幂等”的组合拳，是订房、支付、通知所有异步链路的共同基石——本书反复出现，请刻进肌肉记忆。

【陷阱】用时间戳排序消息——时钟不可靠

新手常用“消息创建时间戳”给会话内消息排序。这在分布式下**不可靠**：多台服务器时钟有漂移、可能回拨，两条消息可能拿到相同甚至倒序的时间戳，导致会话里消息**乱序**。正确做法是为每个会话维护一个**单调递增的序号 seq**（由会话所在分片集中分配，或用逻辑时钟），消息严格按 seq 排序与去重。需要全局唯一且大致有序的 ID 时用**雪花算法（Snowflake）**（时间高位 + 机器 + 序列）。记住：**分布式系统里，永远不要把物理墙钟当作排序或唯一性的依据**，要用逻辑序号 / 单调计数器。

7.1.6 扇出（Fan-out）

一对一消息扇出简单；**系统通知 / 群发**需要扇出给大量接收者。沿用 feed 系统的经验：**写扩散（push/fan-out-on-write）**在发送时就把消息写进每个接收者的收件箱，读快写重，适合普通用户；**读扩散（pull/fan-out-on-read）**读时再聚合，写轻读重，适合超大群/广播。常**混合**：普通会话写扩散，超大广播读扩散。

【追问】追问：如何保证同一会话的消息严格有序，即使并发发送？

要点：(1) **把有序性收敛到单一序列化点**——同一 `conv_id` 的所有消息都路由到**同一个 Kafka 分区**（按 `conv_id` 做 partition key），Kafka 保证单分区内有序消费。(2) seq 的分配也要在单点串行：由该会话归属的分片/服务实例**原子自增**分配 seq，避免并发拿到重复或乱序 seq。(3) 客户端按 seq 排序渲染，对乱序到达的消息按 seq 插入正确位置；若发现 seq 空洞（中间缺号）则触发**拉取补齐**。(4) 跨会话之间无需全局有序，只需会话内有序，这大大降低了难度。核心思想：**把“全局有序”这个昂贵需求缩小为“分区内/会话内有序”，再用单分区单写者来保证**。

7.2 第二部分：支付与信任系统

支付是“一个字节都不能错”的系统。Airbnb 的特殊性在于它是**双边市场**：钱从房客来，要（在合适时机）付给房东，平台居中托管、抽佣、处理退款拒付，还要防双边欺诈。

7.2.1 第一步：需求澄清

功能性：

- 房客支付（信用卡/钱包/多种本地支付方式），**多币种**。
- 平台**托管（escrow）**：钱先到平台，入住后再结算给房东（保护双方）。
- 退款、部分退款、拒付（chargeback）处理。

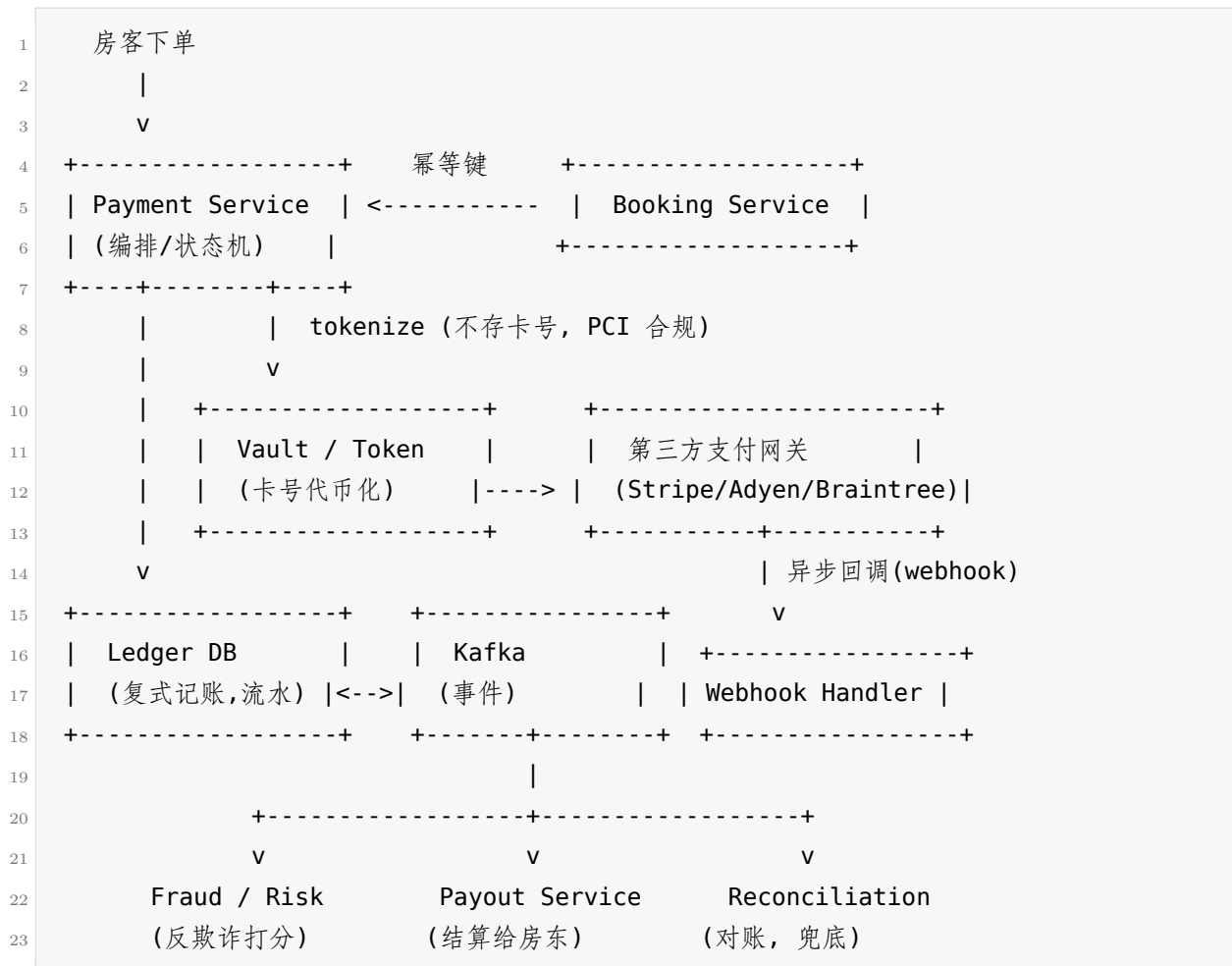
- 对账、流水可审计。
- 反欺诈与信任安全：识别盗卡、虚假房源、刷单。

非功能性：绝对的强一致与正确性、完整可审计、幂等、合规（PCI-DSS，不自存卡号）。

【设计思想】钱的系统：宁可慢、不可错；强一致压倒一切

和搜索“宁可旧不可错地拒绝”不同，支付系统的最高准则是**正确性绝对优先于可用性与延迟**。任何不确定的状态，宁可让交易挂起人工核验，也不能错误地多扣、少付、重复扣款。这意味着：所有金额操作都要**强一致（DB 事务）、幂等、全程留痕可审计**，并以**对账**作为最终防线。设计取舍上，支付系统坦然接受更高的延迟和更复杂的流程来换取“一分钱都不错”。把这条价值观讲出来，本身就是面试加分项。

7.2.2 第二步：高层架构



7.2.3 第三步：核心设计

1) **幂等支付 API**：与订房同理，每次支付请求带**幂等键**；支付服务对同一 key 只执行一次扣款，重复请求返回首次结果。这是防止“网络重试导致重复扣款”的生命线。

```
1 POST /v1/payments
```



```

2 Header: Idempotency-Key: pay_8f3c...e91
3 Body: {
4   "booking_id": 55,
5   "amount_cents": 72000, "currency": "USD",
6   "payment_method_token": "tok_visa_..." # 代币，非真实卡号
7 }
8 -> 201 { "payment_id": 9001, "status": "AUTHORIZED" } # 先授权，后扣款
9 -> 200 (同一 Idempotency-Key 重试) 返回与首次完全相同的结果

```

2) 授权与捕获分离 (auth + capture): 先**授权 (authorize)** 冻结额度，入住前/确认时再**捕获 (capture)** 实际扣款，符合托管语义，也便于失败时直接取消授权而非退款。

3) 复式记账 (double-entry ledger): 用会计的**复式记账**建模资金流——每笔交易至少两条分录，借贷必相等，账目天然自洽、可审计。

【设计思想】复式记账：让钱永不凭空产生或消失

处理金钱，不要用“给余额字段加加减减”，而要用会计沿用数百年的**复式记账 (double-entry bookkeeping)**：每笔资金流动记为成对的分录——一方借 (debit)、一方贷 (credit)，且**系统所有分录之和恒为零**。房客付 720 -> 平台托管账户 +720、房客应付 -720；入住后结算 -> 平台 -650、房东 +650、平台佣金 +70。这样做的威力在于：钱**永远不会凭空产生或消失**，任何不平衡立刻暴露 bug；账本**只追加不修改 (append-only)**，天然可审计、可重放、可对账。这是所有严肃支付/钱包系统的标准建模，远胜于一个可变的 balance 字段。

4) 多币种：金额一律存**最小货币单位的整数 (如分)** + **货币代码**，绝不用浮点数存钱；汇率换算记录**换算时点的汇率快照**，结算与展示分离。

【陷阱】用浮点数存钱——经典且致命

float/double 无法精确表示 0.1 这类十进制小数，累加会产生 $0.1+0.2=0.30000000000000004$ 的误差，在金融场景累积成真实的对不上账。**永远用整数最小单位 (分/厘) 或定点 Decimal 类型存储和计算金额**，货币代码单独存。同时警惕：不同货币最小单位不同 (日元无小数、某些货币三位小数)，汇率换算要**明确四舍五入规则与舍入方向**并留痕，否则双边市场里几分钱的系统性偏差会演变成对账噩梦。

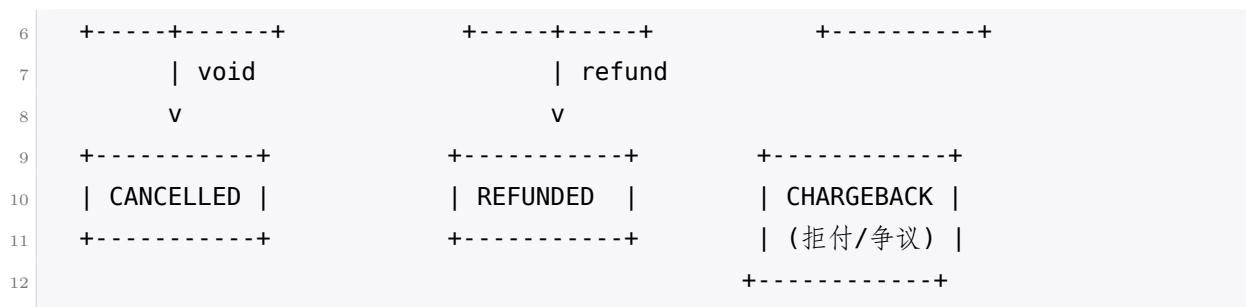
5) 第三方网关集成与 PCI 合规：平台绝不自存原始卡号，把卡号代币化 (tokenize) 交给 Stripe/Adyen 等合规网关，自己只存 token。网关结果常通过**异步 webhook** 回调 (支付是异步的！授权可能几秒后才最终成功/失败)，所以支付状态机必须能处理异步回调与重复回调 (幂等)。

7.2.4 退款、拒付与状态机

```

1   created
2   |
3   v  authorize
4 +-----+ capture +-----+ payout +-----+
5 | AUTHORIZED | -----> | CAPTURED | -----> | SETTLED |

```



每一次状态推进都是带条件的原子更新 + 一条 ledger 分录。退款是反向资金流（同样复式记账），拒付（chargeback）是持卡人向发卡行发起的争议，需要冻结、举证、风控介入。

7.2.5 反欺诈与信任安全

【设计思想】双边市场的信任是核心资产，反欺诈要双向

单边电商只防“买家用盗卡”；Airbnb 这种**双边市场**要同时防**两侧**：房客侧（盗卡支付、拒付欺诈、虚假评价），房东侧（虚假房源、钓鱼、洗钱）。信任一旦崩塌，双边都会流失，所以 trust & safety 是平台的核心资产而非附属功能。设计上：风控以**异步打分 + 同步硬规则**结合——下单链路用轻量同步规则拦截明显欺诈（黑名单、风控分阈值），同时把交易事件送入**异步风控/ML 管线**做深度评估，可疑交易**挂起人工审核**（宁可慢、不可错的又一体现）。把“反欺诈是双向的、信任是资产”讲出来，能体现你理解 Airbnb 的业务本质。

【陷阱】把支付确认同步卡在第三方网关回调上

和订房章的“持锁调支付”同源：不要让用户的下单请求**同步阻塞等待第三方网关返回最终结果**。网关授权可能耗时数秒甚至走异步 webhook，同步等待会拖垮链路、占满连接。正确做法：下单时创建支付记录置为 **PENDING/AUTHORIZING** 立即返回，真正的结果由网关**异步 webhook** 驱动状态机推进；webhook 必须**幂等**（同一事件可能重复回调）且**验签**（防伪造回调）。用户侧展示“支付处理中”，最终态用推送/轮询告知。**所有跨外部网络的调用都应异步化、幂等化，绝不同步阻塞核心链路。**

7.2.6 对账：最终的安全网

无论设计多严密，分布式系统总会有边界情况导致平台账本与第三方网关、银行的记录不一致。**对账 (reconciliation)** 是不可或缺的最终防线：定时拉取网关流水，与本地 ledger 逐笔比对，自动修复可补偿的差异、标记需人工介入的差异。这与订房章的对账、消息系统的幂等去重一脉相承——**异步最终一致系统，必须有一个独立的对账/校验回路来兜底。**

【追问】追问：支付成功但订单确认失败（或反之），钱和订单如何最终一致？

这正是跨服务一致性的核心（呼应订房章 Saga）。要点：(1) **支付服务与订房服务各自持有权威记录**（支付流水 vs 订单状态），通过**事件驱动**异步同步，不强求两库同事务。(2) 关键操作**幂等且可重试**：支付成功事件投到 Kafka，订房侧消费以幂等键保证只生效一次；确认失败则消息不 ack、**重投直至成功**。(3) 用 **Saga 补偿**：若订单最终无法确认（如房源已失效），触

发**自动退款**补偿动作把钱退回，系统回到一致。(4) **对账兜底**：定时扫描“收了钱但订单没确认”“订单确认但没收到钱”的不一致，自动补偿或人工处理。(5) 用户侧呈现“处理中”，避免展示中间态。一句话：**异步消息保最终一致 + 幂等消费 + Saga 补偿 + 对账兜底**，是涉钱系统的标准四件套。

【追问】追问：同一笔支付的 webhook 被重复回调，甚至乱序到达，怎么处理？

要点：(1) **幂等**：每个 webhook 事件有唯一 event_id，处理前先查“是否已处理过该 event_id”，重复的直接返回 200 并跳过副作用——**绝不能因重复回调而重复入账**。(2) **验签**：校验网关签名，拒绝伪造回调（钱的系统尤其要防伪造）。(3) **乱序容错**：webhook 可能乱序（“capture 成功”先于“authorize 成功”到达）。用**状态机的合法转移约束 + 事件时间戳/版本号**处理——只接受合法的状态前进，对“过期/逆向”的事件丢弃或缓存等待前置事件。把状态推进写成**带条件的 CAS** (WHERE status=...)，非法转移自然失败。(4) 对账兜底任何遗漏。本质仍是：**幂等去重 + 状态机约束 + 对账**，与消息系统的不丢不重一脉相承。

【要点】本章速记

- 消息：推拉结合（在线推、离线/上线拉）；at-least-once 投递 + 幂等去重 = 恰好一次效果。
- 会话内有序靠**单调 seq + 同一会话路由到同一分区**，不靠墙钟时间戳；唯一 ID 用雪花算法。
- 已读未读用**单个 last_read_seq 游标**代替海量状态位；扇出按规模选写扩散/读扩散。
- 支付：正确性绝对优先；金额用整数最小单位（绝不用浮点）；幂等支付 API；卡号代币化（PCI 合规）。
- 用**复式记账**让钱不凭空增减、账本只追加可审计；auth/capture 分离支持托管。
- 第三方网关 webhook 必须幂等 + 验签 + 容忍乱序；双边市场反欺诈要双向；**对账**是最终安全网。
- 跨服务一致性统一靠：异步消息 + 幂等消费 + Saga 补偿 + 对账兜底。

Chapter 8

可复用的分布式设计思想

前面几章在具体题目里反复用到了一批“心法”。这一章把它们集中拎出来、独立讲透——因为面试官追问到深处时，考的就是你对这些通用思想的理解。把这章当成一份“设计模式备忘录”。

8.1 幂等性 (Idempotency)

幂等：同一个操作执行一次和执行多次，结果相同。在分布式系统里，网络重试、消息重投、用户重复点击无处不在，幂等是**正确性的地基**。

【设计思想】用幂等键把“最多一次”变成“恰好一次”的体验

实现幂等的通用手段是**幂等键 (idempotency key)**：客户端为一次逻辑操作生成唯一 ID，服务端在处理前先查“这个 key 处理过没有”。第一次处理并记录结果，后续重复请求直接返回上次结果。这样即使底层是“至少一次”投递，用户感知到的也是“恰好一次”。支付、下单、消息消费都靠它。

```
1 处理请求(req):
2     if 存在(req.idempotencyKey):
3         return 取回上次结果(req.idempotencyKey)    # 重复请求，直接返回
4     result = 真正执行(req)
5     保存结果(req.idempotencyKey, result)            # 原子地记录
6     return result
```

【陷阱】“查-再-写”之间的竞态

上面伪代码的“查”和“写”如果不是原子的，两个并发重复请求可能都通过了检查。解决：用数据库唯一约束（把 idempotencyKey 设为唯一索引，第二个插入直接失败），或用 `INSERT ... ON CONFLICT/SETNX` 这类原子原语。**唯一约束是最朴素也最可靠的最后防线。**

8.2 强一致并发控制：悲观锁 vs 乐观锁

“同一资源不能被两个人同时占用”是订房、抢票、库存的共同内核。两种武器：

	悲观锁	乐观锁
做法	SELECT ... FOR UPDATE 先锁住	读时记 version, 写时 CAS 校验
适合	冲突频繁、临界区短	冲突少 (读多写少)
代价	持锁期间阻塞他人、可能死锁	冲突时需重试

【设计思想】乐观锁的本质是“先干活，提交时再验票”

乐观锁不预先加锁，而是假设“多半不会冲突”：读出数据连同版本号，修改后提交时用 UPDATE ... WHERE version = 旧值做原子校验。若影响行数为 0，说明期间有人改过，本次作废重试。它把“串行化”的代价从“所有人排队”降到“只有真正冲突的人重试”，在读多写少场景下吞吐远高于悲观锁。

8.3 Saga 与补偿事务

跨多个服务的操作（下单 -> 扣款 -> 占库存）无法用一个数据库事务包住。**Saga** 把它拆成一串本地事务，每步配一个**补偿动作**；任意一步失败，就反向执行已完成步骤的补偿，把系统拉回一致状态。

- 1 正向：创建订单 -> 冻结房源 -> 扣款 -> 确认订单
- 2 补偿：取消订单 <- 释放房源 <- 退款 <- (无)
- 3 若“扣款”失败 -> 执行“释放房源” -> 执行“取消订单”

【设计思想】用“最终一致 + 补偿”替代“分布式强一致”

两阶段提交（2PC）能给跨服务强一致，但它同步阻塞、协调者单点、性能差，工业界少用。Saga 用**一系列本地事务 + 异步补偿**换取高可用与高吞吐，代价是中间态短暂可见（最终一致）。这正是“把强一致范围收到最小”思想的延伸：只在单个本地事务里要强一致，跨服务用补偿兜底。

8.4 缓存策略与失效

读多写少的系统几乎都靠缓存扛流量。要点：

- **Cache-Aside (旁路缓存)**：读时先查缓存，未命中再查库并回填；写时更新库并删除缓存（而非更新缓存）。
- **失效优于更新**：写后删缓存比写后更新缓存更不容易产生脏数据。
- **三大故障**：缓存穿透（查不存在的 key，用布隆过滤器/空值缓存挡）、击穿（热点 key 过期瞬间打穿，用互斥重建/逻辑过期）、雪崩（大量 key 同时过期，过期时间加随机抖动）。

【陷阱】缓存与数据库的双写不一致

“先写库再删缓存”在极端并发下仍有窄窗口不一致。务实的兜底：给缓存设合理 TTL（最终自愈），关键路径用延迟双删或订阅 binlog 异步失效。面试时点出“这里存在窄窗口，用 TTL + 异步失效兜底”，比假装完美更得分。

8.5 水平分片 (Sharding)

单库扛不住时按某个 key 把数据拆到多库。

- **分片键选择**是成败关键：要让负载均匀、且高频查询能落在单片。订房系统常按 `listingId` 或地域分片。
- **一致性哈希**让扩缩容时只迁移少量数据。
- **热点**：明星房源/大 V 会让单片过载，需二级拆分或单独处理。

8.6 限流与熔断

- **限流**：令牌桶（允许突发）、漏桶（平滑输出）、滑动窗口计数。保护系统不被打垮。
- **熔断**：下游故障时快速失败（打开断路器），避免线程被拖死引发雪崩；半开状态试探恢复。
- **降级**：高峰期关掉非核心功能（如推荐），保住核心链路（下单）。

【要点】一张思想地图

幂等保正确、乐观/悲观锁防并发冲突、Saga 做跨服务一致、缓存扛读、分片扛写与存储、限流熔断保稳定。任何系统设计题，几乎都是这六件工具的排列组合。

Part IV

行为面与冲刺

Chapter 9

行为面与文化匹配

技术再强，行为面挂了一样拿不到 offer——在 Airbnb 这一轮甚至可以一票否决。好消息是：行为面是**唯一可以提前把答案准备到接近完美**的一轮。这一章给你一套可落地的故事框架。

9.1 STAR 框架

每个行为问题，都用 STAR 结构作答，控制在 2-3 分钟：

- **S (Situation)**：背景，一句话交代清楚。
- **T (Task)**：你的具体目标/职责。
- **A (Action)**：你做了什么（重点，占七成篇幅，强调“我”而非“我们”）。
- **R (Result)**：结果，尽量带可量化的数字，并补一句你从中学到了什么。

【设计思想】故事要“可追问”，细节要真

面试官一定会顺着你的故事往下钻：“当时为什么选这个方案？”“别人反对你怎么办？”所以**只讲你真正经历过的事**，把细节（数字、技术选型、冲突的具体对话）想清楚。一个编造的完美故事，撑不过三个追问；一个真实但平凡的故事，反而能越聊越深、越聊越可信。

9.2 对应四条价值观的故事清单

提前准备 6-8 个故事，覆盖下面每条价值观至少一个。一个好故事往往能同时命中多条。

价值观	你需要一个能体现……的故事
Champion the Mission	为了用户/业务目标，超出本职去推动一件事
Be a Host	主动帮助队友、利他、带新人、补别人的坑
Embrace the Adventure	接下没把握的硬任务、从失败中成长、学新技术
Every Frame Matters	对细节较真、把质量做到超出预期、主人翁意识

9.3 高频问题与应答要点

- **最有挑战的项目？** 选一个技术难度 + 协作复杂度都高的，重点讲你的决策与权衡。
- **一次冲突 / 与人意见不合？** 展示你如何用数据和倾听化解，而非压制对方。结局最好是双方都认可的方案。
- **一次失败 / 犯过的错？** 必须真实，重点在复盘与改进，证明你有成长心态（呼应 Embrace the Adventure）。别选无关痛痒的假失败。
- **为什么是 Airbnb？** 把个人经历和 “belong anywhere” 的使命真诚地连起来，别背官网。

【陷阱】行为面三个致命伤

(1) 通篇 “我们”，听不出你个人贡献；(2) 故事太顺、没有冲突和取舍，显得没深度；(3) 抱怨前同事/前公司——这是 Airbnb 文化轮的大忌，再委婉也别踩。

9.4 把你已有的经历改写成故事

你不需要惊天动地的项目。普通的工程经历也能讲出彩，关键是**框对价值观**。给两个改写示例：

【追问】“讲一个你注重细节的例子” (Every Frame Matters)

可用素材：一次性能优化或一个被你揪出的隐蔽 bug。结构：S——某功能上线后有偶发卡顿，大家以为是网络；T——我负责定位；A——我没止步于 “重启就好”，而是抓 trace、逐帧分析、定位到某次多余的主线程渲染；R——p95 延迟下降 X%，并沉淀了一份排查清单给团队。同时命中 “注重细节 + 利他”。

【追问】“讲一个你啃硬骨头的例子” (Embrace the Adventure)

可用素材：一个你之前不熟、硬学下来的领域（如从移动端转去搭一个分布式服务、或自学一整套数学/AI 体系并产出作品）。重点讲面对不确定时你怎么拆解、怎么自驱坚持到产出，结尾落到 “我现在不怕陌生领域了”。

【要点】准备清单

写下 6–8 个故事的 STAR 要点卡片，每张卡片标注它能命中的价值观；对着镜子或找人各讲一遍，确保每个故事能从容应对至少三个追问。

Chapter 10

资源清单与两周冲刺

最后一章是纯操作手册：免费资源 + 一份可以照着执行的两周计划。

10.1 免费资源清单

10.1.1 系统设计

- **System Design Primer** (GitHub: [donnemartin/system-design-primer](#)): 免费的系统设计圣经，必读。
- **Hello Interview** ([hellointerview.com](#)): 目前口碑最好的系统设计教程，含 Airbnb 类题，部分免费。
- **Bilibili / YouTube**: 搜 “Airbnb 系统设计” “设计订房系统”，有手把手带做的视频。

10.1.2 算法刷题

- **LeetCode** ([leetcode.cn](#)): 按公司标签筛 Airbnb；本书第二部分的十道题都给了编号。
- **免费公司题库**(GitHub): 搜 [krishnadey30/LeetCode-Questions-CompanyWise](#)、[liquidslr/leetcode](#) 等仓库，CSV 形式整理了各公司高频题，免费替代付费题单。

10.1.3 真实面经

- **一亩三分地 (1point3acres)**: 北美中文面经最全，搜 Airbnb VO，最近一年的帖子信息密度最高（部分需积分）。
- **Glassdoor**: 搜 Airbnb -> Interview 标签，真实 onsite 题，免费需登录。
- **LeetCode 讨论区 / 牛客网**: 搜 “Airbnb” 看候选人复盘。

10.2 两周冲刺计划

假设你每天能投入 2-3 小时。原则：算法重质不重量、系统设计吃透少数核心、行为故事尽早动笔。

10.2.1 第一周：建立框架

天 任务

- D1 通读本书第一、二部分；做 LeetCode 68（文本对齐）、341（嵌套迭代器）
- D2 做 269（火星词典）、39（组合总和）；复盘拓扑排序与回溯模板
- D3 做 981（时间键值）、295（数据流中位数）；总结哈希 + 二分、对顶堆
- D4 做 336（回文对）、773（滑动谜题）；练状态编码 BFS
- D5 做 755（倒水）、分页去重（OOD）；重点练“可扩展代码”习惯
- D6 读系统设计方法论 + 设计思想章；手绘订房系统架构图一遍
- D7 吃透“设计订房系统”（防超卖）：默写三种并发方案与取舍

10.2.2 第二周：查漏 + 模拟

天 任务

- D8 吃透“设计搜索系统”：geo 索引（S2/GeoHash）+ 分层缓存
- D9 吃透“设计消息/支付系统”：幂等、Saga、反欺诈
- D10 写 6-8 个 STAR 行为故事，对应四条价值观
- D11 重做前一周做错/不熟的算法题（只挑薄弱的，不贪多）
- D12 全真模拟：找人或自录，1 轮算法 + 1 轮系统设计
- D13 全真模拟：1 轮行为面；逐个故事压三个追问
- D14 总复盘：过一遍本书所有【设计思想】【陷阱】框；早睡

【设计思想】冲刺期最重要的一件事

不是多刷题，而是把少数核心题真正吃透到能讲给别人听。Airbnb 的题会变形、会追加需求，背答案没用；但如果你能把“防超卖的乐观锁”“对顶堆维护中位数”这些思想讲透，再变形的题你也能现场推导出来。质量永远大于数量。

【要点】面试当天

提前测好设备与共享编辑器；准备好纸笔和水；保持“边写边说”的协作姿态；不会时大方说思路、请求提示，而不是沉默死磕。祝你拿到 offer。